ALPHA MICROSYSTEMS
RIGHT. FROM THE START.

AlphaBASIC®
XCALL Subroutine
User's Manual

DSM-00066-03

© 1997 Alpha Microsystems

REVISIONS INCORPORATED	
REVISION	DATE
00	September 1988
01	March 1991
02	September 1996
03	November 1997

AlphaBASIC XCALL Subroutine User's Manual
To re-order this document, request part number DSO-00066-00.

This document applies to AMOS version 2.3A, PR 12/97 and later.

The information contained in this manual is believed to be accurate and reliable. However, no responsibility for the accuracy, completeness or use of this information is assumed by Alpha Microsystems.

This document may contain references to products covered under U.S. Patent Number 4,530,048.

The following are registered trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AMIGOS	AMOS	Alpha Micro	AlphaACCOUNTING
AlphaBASIC	AlphaCALC	AlphaCOBOL	AlphaDDE
AlphaFORTRAN 77	AlphaLAN	AlphaLEDGER	AlphaMAIL
AlphaMATE	AlphaNET	AlphaPASCAL	AlphaRJE
AlphaWRITE	CASELODE	OmniBASIC	VER-A-TEL
VIDEOTRAX			

The following are trademarks of Alpha Microsystems, Santa Ana, CA 92799:

AlphaBASIC PLUS	AlphaVUE	AM-PC	AMTEC
AlphaDDE	AlphaConnect	DART	*in*Sight/am
*in*Front/am	ESP	MULTI	

All other copyrights and trademarks are the property of their respective holders.

ALPHA MICROSYSTEMS
2722 S. Fairview St.
P.O. Box 25059
Santa Ana, CA 92799

Table of Contents

CHAPTER 1 - INTRODUCTION 1-1

WHAT IS A SUBROUTINE? 1-1
MANUAL ORGANIZATION 1-1
THE SAMPLE PROGRAMS IN THIS BOOK 1-2
USING XCALL SUBROUTINES 1-3
GRAPHICS CONVENTIONS 1-4

CHAPTER 2 - BASORT SORTS FILES 2-1

WHAT DOES BASORT DO? 2-1
HOW TO LOAD BASORT INTO MEMORY 2-2
HOW TO USE BASORT TO SORT RANDOM FILES 2-2
Random File Example 2-3
HOW TO SORT SEQUENTIAL FILES 2-5
Sequential File Example 2-6

CHAPTER 3 - COMMON PROVIDES COMMON VARIABLE STORAGE 3-1

WHAT IS COMMON? 3-1
HOW TO LOAD COMMON INTO USER OR SYSTEM MEMORY 3-1
HOW TO USE COMMON 3-2
Defining Variables 3-2
COMMON Format 3-3
An Example Of COMMON 3-3

CHAPTER 4 - FLOCK CO-ORDINATES MULTI-USER FILE ACCESS 4-1

WHAT IS FLOCK? 4-1
THE MULTIPLE UPDATE PROBLEM 4-1
WHAT IS AN INTERCONSISTENCY PROBLEM? 4-2
LOADING FLOCK INTO SYSTEM MEMORY 4-2
FLOCK FORMAT 4-2
Action and Mode 4-2
File 4-3
Record 4-4
Return-code 4-4
QUEUE BLOCK REQUIREMENTS 4-4
HOW TO USE FLOCK 4-5
File-Open Interlocks 4-6
Record-Update Interlocks 4-6
Improved File Interlocks 4-7

DEADLOCK, AND HOW TO PREVENT IT 4-7
QUICK REFERENCE SUMMARY OF ACTIONS/MODES 4-8

CHAPTER 5 - XLOCK SETS AND TESTS DEVICE AND FILE LOCKS 5-1

WHAT IS XLOCK? 5-1
LOADING XLOCK INTO SYSTEM MEMORY 5-2
THE XLOCK FORMAT 5-2
THE LOCKS 5-3
THE MODES 5-3
Mode 0 (Lock and Return) 5-4
Mode 1 (Lock and Wait) 5-4
Mode 2 (Clear Lock) 5-4
Mode 3 (List Locks) 5-4
WILDCARDS 5-5
PROGRAMMING EXAMPLES 5-5
Calculating Record Numbers 5-6
Sample Program to Illustrate File Record Locking 5-7

CHAPTER 6 - SPOOL PRINTS FILES 6-1

WHAT IS SPOOL? 6-1
TRADITIONAL SPOOL FORMAT 6-1
TASK MANAGER SPOOL FORMAT 6-2
HOW TO USE SPOOL 6-3

CHAPTER 7 - XMOUNT MOUNTS A DISK 7-1

WHAT IS XMOUNT? 7-1
XMOUNT FORMAT 7-1
HOW TO USE XMOUNT 7-2

CHAPTER 8 - GTLANG RETURNS LANGUAGE DEFINITION TABLE 8-1

WHAT IS GTLANG? 8-1
GTLANG FORMAT 8-1
HOW TO USE GTLANG 8-3

CHAPTER 9 - TRMCHR GETS TERMINAL CHARACTERISTICS 9-1

WHAT IS TRMCHR? 9-1
TRMCHR FORMAT 9-1
HOW TO USE TRMCHR 9-3

CHAPTER 10 - TERMINAL INPUT AND DISPLAY SUBROUTINES 10-1

ACCEPT 10-1
ECHO 10-1
NOECHO 10-1

CHAPTER 11 - MISCELLANEOUS XCALLS 11-1

IDTIM 11-1
ODTIM 11-2
Ordering and Formatting Options 11-2
RENAME 11-5
SLEEP 11-6
STRIP 11-6
MSGLOG 11-6
LKINFO 11-7

CHAPTER 12 - FNDUSR LOCATES A USER DESCRIPTION 12-1

WHAT IS FNDUSR? 12-1
FNDUSR FORMAT 12-1
The User'map Fields 12-2

CHAPTER 13 - TCP TCP NETWORKING IN ALPHABASIC 13-1

TCP PROGRAMMING OVERVIEW 13-1
Client/Server Paradigm 13-1
IP Addresses and Port Numbers 13-1
Service Names 13-2
Host and Domain Names 13-2
TCP XCALL OVERVIEW 13-3
Procedural Versus Event-Driven 13-3
Features 13-3
Program Operation 13-4
Variable Types 13-4
TCP XCALL PROGRAMMING 13-4
Include File 13-5
Detecting Events 13-5
Handles 13-6
Process IDs 13-6
Events 13-7
Error Codes 13-8
Establishing Connections 13-8
Accepting Connections - Part One 13-10
Accepting Connections - Part Two 13-11
Sending Data 13-11
Reading Data 13-12
Closing Sessions 13-13
Spawning Jobs 13-13

Terminating Spawned Jobs 13-14
Important Cleanup 13-14
Attention Events 13-14
Transfer of Control 13-15
Path and Filename Conversion 13-15
Connection Information 13-16
Setting the Alarm Clock Timer 13-16
Trapping SIEXIT Requests 13-16
QUICK REFERENCE LIST OF CALLS 13-17
TAME ERROR MESSAGES 13-18
General Error Codes 13-19
Socket Errors 13-19
Name (DNS Resolver) Errors 13-20
PROGRAMMING SAMPLES 13-20

APPENDIX A - QUICK REFERENCE A-1

ACCEPT A-1
BASORT A-1
COMMON A-2
ECHO A-2
FLOCK A-2
FNDUSR A-2
GTLANG A-3
IDTIM A-3
LKINFO A-3
NOECHO A-3
ODTIM A-3
RENAME A-4
SLEEP A-4
SPOOL A-4
STRIP A-5
TCP A-5
TRMCHR A-5
XLOCK A-6
XMOUNT A-6

APPENDIX B - ERROR MESSAGES B-1

APPENDIX C - GLOSSARY C-1

DOCUMENT HISTORY

INDEX

Chapter 1 - Introduction

This chapter discusses:

- What is a subroutine?
- How is this manual organized?
- What do our sample programs look like?
- How do you use an XCALL?

WHAT IS A SUBROUTINE?

A subroutine is a section of computer code that performs a function. When a subroutine is "called" by a program, it is executed as if it were part of the program. A subroutine saves a programmer from having to re-write a section of commonly-used program code.

AlphaBASIC, the Alpha Micro BASIC Language Processor, is a powerfully enhanced version of BASIC that has the ability to access external subroutines that are written in machine language, using a keyword called XCALL. Several machine language subroutines, ones that perform complex and yet frequently required tasks, are provided on your System Disk. These external subroutines, their features, abilities and restrictions, are the subject of this manual.

Because these external subroutines are machine language programs, they are much smaller and faster than equivalent AlphaBASIC programs. Machine language programs work closely with hardware and the operating system, which AlphaBASIC cannot do in some applications.

It is important to note here that, whereas you can write your own machine language subroutines and access them by using the XCALL statement, this manual does not discuss how those machine language subroutines can be written. See your *AlphaBASIC User's Manual* for information. This manual instead restricts itself to a discussion of the existing external subroutines, and how you can use them in your own AlphaBASIC programs.

You will find this manual useful if you are already somewhat familiar with AlphaBASIC and wish to understand, and then access, these external subroutines. You may also find this manual to be useful later as a reference guide to the various existing subroutines (see Appendix A, the Quick Reference Guide). Appendix C contains a glossary of terms used in this manual. See your *AlphaBASIC User's Manual* for more information about AlphaBASIC.

MANUAL ORGANIZATION

This manual is arranged in chapters. Each chapter discusses one or more of the subroutines. Here is a quick list of the subjects of each chapter:

Chapter	Subject
2	BASORT, which sorts random and sequential files
3	COMMON, which transfers data between programs
4	FLOCK, which protects data in files and records
5	XLOCK, which sets, tests, and clears “locks”
6	SPOOL, which sends files to a printer
7	XMOUNT, which mounts a disk
8	GTLANG, which gives you information about the language currently set for your job
9	TRMCHR, which returns terminal information
10	Terminal input and display XCALLs
11	Miscellaneous XCALLs
12	FNDUSR, which returns information about specific users
13	TCP, which gives your programs access to the TAME (TCP Access Made Easy) programming interface

THE SAMPLE PROGRAMS IN THIS BOOK

There are a number of sample programs in this manual, ranging in complexity from one to several dozen program lines. These samples are included to demonstrate the use of the AlphaBASIC XCALL subroutines, and may not represent the best way to write AlphaBASIC programs.

To quickly grasp the point of these examples, remember that AlphaBASIC permits the use of labels, as well as line numbers, to identify locations in a program. A program label is composed of one or more alphanumeric characters which are not separated by a space or other delimiter. The first character is always a letter. A label must be the first item on a line after the optional line number and must be terminated by a colon (:). Here is an example of labels (RANDOM’DIRECTION, UP, DOWN and STRAIGHT) in an AlphaBASIC program that performs a kind of simple animation:

```
MAP 1  DIRECTION,F
RANDOM’DIRECTION:
  DIRECTION = INT(3 * RND(0) + 1)
  ON DIRECTION GOSUB UP, DOWN, STRAIGHT
  GOTO RANDOM’DIRECTION
UP:          PRINT “/”; TAB(-1,3);
             RETURN
DOWN:        PRINT TAB(-1,4);”\”;
             RETURN
STRAIGHT:    PRINT”--”;
             RETURN
```

Note the first line of the above program example is a level 1 MAP statement; we map the variable DIRECTION as a floating point variable (F).

AlphaBASIC provides you with the ability to specify the pattern in which variables of all kinds (floating point, string, and binary) are allocated in memory. By mapping variables at different levels you may define whole groups of related information and reference single elements or an entire group as you choose.

For further information on interpreting and using MAP statements, see your *AlphaBASIC User's Manual*.

USING XCALL SUBROUTINES

There are several things you should keep in mind before beginning to use XCALL subroutines:

- All XCALL subroutine files must have a .SBR extension. The subroutines supplied with your system software are in account [7,6] of the System Disk (DSK0:).
- Whenever your program requests a subroutine, AlphaBASIC follows a specific pattern in looking for the requested subroutine. The search sequence is:
  1. System memory.
  2. User memory.
  3. Your current account.
  4. The ,0 account in your current project. For example, if you are logged into DSK2:[100,5], the account DSK2:[100,0].
  5. The BASIC account, DSK0:[7,6].

  Notice AlphaBASIC checks first system, then user memory. If a subroutine is to be called a large number of times, it is wise to load it into memory to avoid the time spent in fetching the subroutine from the disk.

  If the subroutine is not in memory, AlphaBASIC attempts to load the subroutine from the disk. If an AlphaBASIC program gets a subroutine from the disk, AlphaBASIC loads it into memory only for the duration of its execution.

  Subroutines loaded into system or user memory by the LOAD command remain in memory until you reset the system or until you use the monitor command DEL to delete them. However, some subroutines have restrictions—see the individual XCALL description for exactly where and when to load them.
- Your AlphaBASIC program calls and runs a subroutine by using the XCALL statement, and you will usually need to add control parameters to the XCALL statement line.
- If you need to use binary numbers with the subroutine, you will need to define them first by using a MAP statement in your AlphaBASIC program.
- Many of the subroutines require you to pre-load special files. For each subroutine, the documentation that follows will let you know what files need to be pre-loaded. To load a subroutine file into user memory (your own memory partition), enter the following from AMOS command level:

  ```
  LOAD DSK0:filename.SBR[7,6] RETURN
  ```

  filename is the name of the subroutine you are requesting (COMMON, BASORT, etc.). After you see the monitor prompt, you may run an AlphaBASIC program that uses that specific subroutine.

  To load an XCALL subroutine into system memory, the System Operator must include a SYSTEM command within the system initialization command file. For more information on loading files, including subroutines, into system memory during system bootup, see your *System Operator's Guide*.

GRAPHICS CONVENTIONS

This manual conforms to the other Alpha Micro publications in its use of a standard set of graphics conventions. We hope these graphics simplify our examples and make them easier for you to use. Unless stated otherwise, all examples of commands are assumed to be entered at AMOS command level.

SYMBOL	MEANING
devn:	Device Name. The "dev" is the three letter physical device code, and the "n" is the logical unit number. Examples of device names are DSK0:, DSK5:, WIN1:, and MTU0:. Usually, device names indicate disk drives, but they can also refer to magnetic tape drives and CD-ROMs.
filespec	File Specification. A file specification identifies a specific file within an account. A complete filespec is made up of the device name, the file name, the file extension, and the project-programmer number, in the format `devn:filename.ext[p,pn]` For example: `DSK0:SYSTEM.INI[1,4]`
[p,pn]	Project-Programmer Number. This abbreviation represents an account on a disk you can store files and data in. An actual disk account number looks like this: [100,2] or [1,4]. Disk account specifications are sometimes referred to as "Project-programmer numbers."
{ }	Braces are used in some examples to indicate optional elements of a command line. In the example: DIR{{/switch} the braces tell you "/switch" is not a required portion of the DIR command line.
/	The slash precedes a command line switch or "option request." For example: DIR/WIDE:3 [RETURN] This command requests a directory display of the disk account you are currently logged into. The switch (/WIDE:3) indicates you want the display to be three columns wide.
TEXT	Bold text in an example of user/computer communication represents the characters you type.
TEXT	Text like this in an example of user/computer communication represents the characters the computer displays on your screen. We also use this typeface for programming statement format and examples. In statement format, upper case letters must be entered exactly as shown; variable parts of the statement (such as variable names) are in lower case.
reference	We use bold italic type to refer to parts of an XCALL statement's format. The first reference is always in this typeface; further references use it only if they might be unclear otherwise.
[KEY]	In our examples, the keycap symbol appears whenever you need to press a certain key on your terminal keyboard. The name of the key you need to press appears inside the keycap symbol, like this: [RETURN]. If you need to press the TAB key, you would see [TAB], or the ESCAPE key, [ESC]. (Sometimes the ESCAPE key is labeled ESC or ALT MODE.)

SYMBOL	MEANING
CTRL / KEY	This indicates a control sequence you press on the keyboard. Hold down the CTRL key while you press the indicated key.
^	This symbol in front of a capital letter means the letter is a “control character.” For example, when you press CTRL /C, it appears on your screen as ^C. (^C is the control character that cancels most programs and returns you to AMOS command level.)
	This symbol means “halt!” It indicates an important note you should read carefully before going further in the documentation. Usually, text next to this symbol contains instructions for something you MUST or MUST NOT do, so read it carefully.
	This symbol means “hint.” It indicates a helpful bit of information, or a “short cut” that could save you time or trouble.
	This symbol means “remember.” It indicates something you should keep in mind while you are following a set of instructions.

Chapter 2 - BASORT Sorts Files

This chapter discusses:

- What does BASORT do?
- How to load BASORT into memory
- Using BASORT to sort a Random file
- Using BASORT to sort a Sequential file

WHAT DOES BASORT DO?

BASORT sorts both random and sequential files. A random file is one in which the records are physically grouped together in one area of the disk, and where any point within that file can thus be found immediately by calculating an offset from the file's beginning.

A sequential file's records are not necessarily contiguous on the disk, but are linked in sequence by pointers in each segment indicating where on the disk the next segment can be found. For information on creating and using files from within AlphaBASIC, see your *AlphaBASIC User's Manual.*

You can use BASORT to sort the contents of a file into numeric order, a list of names or words into alphabetic order, and so on. BASORT permits the use of up to three keys, or elements of the data records you wish to base your sort on.

For example, say you have a list of customer names, each with an associated order date code and a purchase order number. The first key might be the customer name. If a particular customer has ordered more than once, the second key comes into play to determine which record of that customer's should go first. You can sort that customer's orders chronologically based on the date code. And if that customer has placed two or more orders in the same day, the third key will determine the final sorting placement of that customer's records based on his purchase order numbers.

BASORT combines two sorting methods to make it a relatively fast sort utility that can still handle very large files. If your memory partition is large enough to contain the entire file to be sorted, BASORT performs a memory-based sort. That means it sifts through and re-arranges data in memory to bring the data into the order you specified in the BASORT command line.

If there is not enough room in user memory for the entire file, BASORT does a disk-based polyphase merge-sort. That is, the data is brought into memory in small groups where it is sorted and re-written to the disk; then the several groups are merged together on the disk.

HOW TO LOAD BASORT INTO MEMORY

The BASORT package consists of two modules—BASORT.SBR and AMSORT.SYS. These modules must be in memory when BASORT is used. When the XCALL BASORT command is used in an AlphaBASIC program, the AlphaBASIC program automatically loads BASORT.SBR into user memory. However, AMSORT.SYS must be loaded into either system or user memory prior to running an AlphaBASIC program that uses BASORT.

To load AMSORT.SYS into user memory, enter:

LOAD DSK0:AMSORT.SYS[1,4] RETURN

To load AMSORT.SYS into system memory, you must have a line in your system initialization command file like this:

```
SYSTEM DSK0:AMSORT.SYS[1,4]
```

For more information on loading subroutines into system memory during system boot-up, see your *System Operator's Guide*.

HOW TO USE BASORT TO SORT RANDOM FILES

When you use BASORT to sort random files, BASORT sorts the file onto itself (that is, it replaces the original, unsorted file with a file containing the sorted data). Therefore, if you wish to keep a backup copy of the unsorted file, you must create a separate copy to be sorted.

The format for using BASORT on random files is:

```
XCALL BASORT, Channel'Number, Record'Count, Record'Size, &
      Key1'Size, Key1'Position, Key1'Order, &
      Key2'Size, Key2'Position, Key2'Order, &
      Key3'Size, Key3'Position, Key3'Order, &
      Key1'Type, Key2'Type, Key3'Type
```

Channel'Number	File channel on which file to sort is open for random processing.
Record'Count	Number of records in the random file you are sorting.
Record'Size	Size of the longest record in the file you are sorting. The size of a record is the number of bytes in that record (including characters, spaces, etc.).
Key1'Size	The size, in bytes, of sort key #1. Give the size of the largest instance of key #1 (i.e., if sort key #1 is the customer's name, find the longest name in any record, or perhaps allow for a very long one).
Key1'Position	The first character position occupied by key #1. If the Key1'Position variable given is 50, for example, BASORT will fit the characters beginning at the fiftieth byte in the record into the sequence it is creating.

Key1'Order	Sort order of key #1. Enter the digit 0 to indicate you want key #1 of each record to be sorted in ascending sequence, or enter 1 for descending sequence. **The order is determined using ASCII collating sequence—all upper-case letters come before lower-case letters.**
Key2'Size	The size, in bytes, of sort key #2.
Key2'Position	The first character position of sort key #2.
Key2'Order	Sort order of key #2. Enter a 0 or a 1. (See Key1'Order, above.)
Key3'Size	The size, in bytes, of sort key #3.
Key3'Position	The first character position of sort key #3.
Key3'Order	Sort order of key #3. Enter a 0 or a 1. (See Key1'Order, above.)
Key1'Type	The data type of key #1. Key types are: 0 = String, 1 = Floating Point, 2 = Binary.
Key2'Type	The data type of key #2. See Key1'Type.
Key3'Type	The data type of key #3. See Key1'Type.

Remember, keys are the elements of the data records you want to base your sort on (i.e., customer name, order number, etc.). If you want to use fewer than three keys, all entries in the XCALL command line for the unused keys must be zero. If the key types are omitted, BASORT assumes string data type.

All arguments in the XCALL command line are numeric, but may be passed as either floating point or string values. For example, "99" is a valid entry. Arguments must NOT be in binary format.

The first character in a record is considered position 1.

Random File Example

The following is an unsorted file we'll use as an example. We'll call the file PO.DAT. It contains customer names, dates, and purchase order numbers:

```
ROBIN GOOD PUBLICATIONS                  1/3/89     49130
ANDERSON'S ANACHRONISMS                  12/7/89    1207
EVANS' CLASSIC AUTOMOBILES, INC.         1/20/89    K79876
ROBIN GOOD PUBLICATIONS                  2/14/89    49201
DE SOTO HORSE GROOMING EQUIPMENT CO.     4/9/89     1895
TOMLINSON'S NEEDLE ARTS                  8/3/89     14101
BOB'S AIRPLANE RIDES                     2/28/89    49393
BRISCOE'S GRAPHICS                       6/12/89    7S729
HONEST DAVE'S CHEAP CAR PARTS            9/11/89    A00326
DE SOTO HORSE GROOMING EQUIPMENT CO.     4/9/89     1895
ROBIN GOOD PUBLICATIONS                  2/28/89    49397
EVANS' CLASSIC AUTOMOBILES, INC.         9/11/89    L98467
```

The program we will use to sort the above file is:

```
! Sample program to sort a random data file:
!
MAXNUM = 25
MAP1  CUSTOMER'INFO              ! Definition of the Record:
      MAP2  NAME,S,35            !   35 bytes maximum
      MAP2  PURCHASE'DATE,S,8    !    8 bytes maximum
      MAP2  PURCHASE'ORDER,S,7   !    7 bytes maximum
MAP1  RECORD'SIZE,F,6,50         !   50 bytes maximum
MAP1  RECORD'NUMBER,F,6,0        ! First record is #0
MAP1  CHANNEL,F,6,100            ! Open channel is #100
MAP1  RECORD'TOTAL,F,6,12        ! Total of 12 records
MAP1  ASCENDING,F,6,0            ! Sort in ascending order
MAP1  STRING,F,6,0               ! All keys are strings

START:
  OPEN #100,"PO.DAT",RANDOM,RECORD'SIZE,RECORD'NUMBER
  PRINT "Now sorting..."
  XCALL BASORT,CHANNEL,RECORD'TOTAL,RECORD'SIZE,35,1, &
        ASCENDING,8,36,ASCENDING,7,44,ASCENDING,STRING, &
        STRING,STRING
  PRINT "We will sort on name, purchase date,"
  PRINT "and purchase order number:" : PRINT
  FOR RECORD'NUMBER = 0 TO MAXNUM
      READ #100,CUSTOMER'INFO
      PRINT NAME,
      PRINT PURCHASE'DATE,
      PRINT PURCHASE'ORDER
      IF EOF(100) = 1 THEN RECORD'NUMBER = MAXNUM
  NEXT
  CLOSE #100
END
```

Note the line right after `START:` that opens the file, PO.DAT. The XCALL statement then calls the BASORT subroutine, where the variables (defined in the MAP statements) define the BASORT parameters. The file is sorted back on itself at that point. Then it is printed, and the file is closed.

The resulting printout looks like this:

```
Now sorting...
We will sort on name, purchase date,
and purchase order number:

ANDERSON'S ANACHRONISMS                  12/7/81   1207
BOB'S AIRPLANE RIDES                     2/28/81   49397
BRISCOE'S GRAPHICS                       6/12/81   7S729
DE SOTO HORSE GROOMING EQUIPMENT, Co.    4/9/81    1836
DE SOTO HORSE GROOMING EQUIPMENT, Co.    4/9/81    1895
EVANS' CLASSIC AUTOMOBILES, INC.         1/20/81   K79876
EVANS' CLASSIC AUTOMOBILES, INC.         9/11/81   L98467
HONEST DAVE'S CHEAP CAR PARTS            9/11/81   A00326
ROBIN GOOD PUBLICATIONS                  1/3/81    49130
ROBIN GOOD PUBLICATIONS                  2/14/81   49201
ROBIN GOOD PUBLICATIONS                  2/28/81   49393
TOMLINSON'S NEEDLE ARTS                  8/3/81    14101
```

HOW TO SORT SEQUENTIAL FILES

When you sort a sequential file, you must specify both an input and an output file. If you wish to sort a file back onto itself, you may specify the same file for both input and output.

Before BASORT is called, the file must be opened for input. BASORT leaves the file open for output. Remember to load AMSORT.SYS into memory before using BASORT.

The format for calling BASORT for sequential files is:

```
XCALL BASORT, Input'Channel, Output'Channel, Record'Size, &
      Key1'Size, Key1'Position, Key1'Order, &
      Key2'Size, Key2'Position, Key2'Order, &
      Key3'Size, Key3'Position, Key3'Order
```

Input'Channel	The file channel on which the input file is open.
Output'Channel	The file channel on which the output file is open.
Record'Size	The size, in bytes, of the largest record in the file, including the terminating carriage return/linefeed characters. **Too small a value results in truncation of data records.**
Key1'Size	The size, in bytes, of sort key #1. Give the size of the largest instance of key #1 (i.e., if sort key #1 is the customer's name, find the longest name in any record, or perhaps allow for a very long one).
Key1'Position	The first character position of sort key #1. If the Key1'Position variable given is 50, for example, BASORT will fit the characters beginning at the fiftieth byte in the record into the sequence it is creating.
Key1'Order	Sort order of key #1. Enter the digit 0 to indicate you want to sort key #1 in ascending sequence, or enter the digit 1 to choose descending sequence. **The order is determined using ASCII collating sequence—all upper-case letters come before lower-case letters.**
Key2'Size	The size, in bytes, of sort key #2.
Key2'Position	The first character position of key #2.
Key2'Order	Sort order of key #2. Enter a 0 or a 1. (See Key1'Order, above.)
Key3'Size	The size, in bytes, of sort key #3.
Key3'Position	The first character position of key #3.
Key3'Order	Sort order of key #3. Enter a 0 or a 1. (See Key1'Order, above.)

Sequential files contain only ASCII data. For that reason, when you sort sequential files you do not have to specify the data type of the sort keys; BASORT knows all keys in a sequential file are strings.

Sequential File Example

The following is the contents of a sequential file we want to sort. Suppose this time we are cartographers, making a map of a new suburb. We want to compile an alphabetic index of the streets laid out and defined so far, and we also want an alphabetic list of the streets to the north of town center only, then one of the streets to the east, and so on for the streets to the south and west.

We use a sequential file for this data because as new streets are laid out and named, we can add them to our sequential file and then re-sort the file for future maps.

We have gathered the existing street names and their relative positions from city plans. The file containing the unsorted list of streets is called STREET.DAT. The extension, .DAT, indicates to us this is the raw data file.

We want to record the sorted, alphabetic list of all the streets in a file called STREET.LST. The street names sorted according to direction we'll place in a file called ENSW.LST.

We choose an extension of .LST to remind us these are files we can print when we want to. Here is the list of street names and directions we've gathered from city plans:

```
Sinbad Street            N
John Silver Road         W
Marco Polo Avenue        E
Robinson Crusoe Drive    S
James Bond Court         S
William Tell Lane        W
Achilles Drive           W
Harold Hardrede Court    N
Mick Dundee Street       E
Kublai Khan Circle       S
Darth Vader Road         W
Sancho Panza Circle      E
Jason Bourne Road        N
John Carter Lane         N
Sherlock Holmes Avenue   E
Robin Hood Avenue        S
Edward Teach Street      E
Cisco Kid Road           S
Michael Finn Drive       N
Ernest Hemingway Lane    W
```

The first thing we need to do is load AMSORT.SYS into user memory. We do that at AMOS command level, this way:

LOAD DSK0:AMSORT.SYS[1,4] RETURN

Now we create the AlphaBASIC program. The first thing we have to remember to do is open the file channel for the file we want to sort, and two more file channels and files where we want to put the sorted data (or we could use just one other file if we wanted to write over the original, unsorted data). Our program might look like this:

```
! Sample program to sort a sequential data file:

START:
  OPEN #1,"STREET.DAT",INPUT
  OPEN #2,"STREET.LST",OUTPUT
  OPEN #3,"ENSW.LST",OUTPUT
  PRINT "Now sorting all streets alphabetically."
  XCALL BASORT,1,2,50,25,1,0,1,30,0,0,0,0
  CLOSE #1
  PRINT "Now sorting according to the direction ";
  PRINT "from the center of town."
  OPEN #1,"STREET.DAT",INPUT
  XCALL BASORT,1,3,50,1,30,0,25,1,0,0,0,0
  PRINT "All done." : PRINT
  PRINT "See STREET.LST and ENSW.LST for sorted files."
  CLOSE #1 : CLOSE #2 : CLOSE #3
END
```

The first OPEN statement opens file channel #1 and the file called STREET.DAT for input. The next opens file channel #2 and the file called STREET.LST for output. The next opens file channel #3 for output also.

We then call the first XCALL BASORT subroutine. Immediately following the word BASORT and the delimiting comma, we indicate the file channel open for input. Then we indicate the output file channel, 2, where we want the file sorted the first way to be stored.

Note in the unsorted file above, a record (the data of a single street name) is confined to one line. That makes it easy to judge the approximate size of the longest record. So, being liberal, we round it up to a record-size of 50.

Key #1 is never more than 25 bytes in size, so next on the XCALL BASORT line we enter a 25. The position of Key #1 is the first byte in the record (column 1, as it happens), so we enter a 1. Next, we must specify a 0 or a 1 to flag whether we want to sort Key #1 in ascending or descending order. Our street index is alphabetically ordered, (starting at A and ending at Z), so we enter a 0 here to choose ascending order.

Key #2 is our direction, N, S, E or W. The size of Key #2 in this case is always 1. The position of Key #2 in our file STREET.DAT is column (or record byte number) 20. We don't really care whether our directions are ascending or descending yet, but we'll enter a 0 to indicate ascending order.

We don't have a Key #3, so we specify Key #3 size, position and order as 0, 0, and 0 respectively.

Note we don't specify the data type of keys #1, #2 and #3 for a sequential file because they are always ASCII data, which BASORT knows.

Next, the file STREET.LST is created and the data in STREET.DAT is rewritten in alphabetical order. The next lines are in the program to close and then re-open file channel #1 and the file STREET.DAT. If those two lines are omitted, the new file ENSW.LST, though created, would be empty because no FURTHER data would be found in the file STREET.DAT. These two lines cause the BASORT subroutine to look at the beginning of the file, rather than the end.

We again call the BASORT program. This line is different than the first because we are now specifying the direction byte (N, S, E or W) as Key #1 and the street name as Key #2.

The size of key #1 is always just 1 byte in size, so on the XCALL BASORT program line after the delimiting comma following the subroutine name, we enter a 1. The position of Key #1 is the twentieth byte (column 20), so we next enter a 20. We must specify a 0 or a 1 to flag whether we want to sort Key #1 in ascending or descending order. ASCII sequence puts E first, then N, then S and W, which is fine with us. We'll enter a 0 here.

Key #2 this time is the street name. That is, the size of Key #2 in this case is 25. The position of Key #2 is column (or record byte number) 1. We want the street names within the four direction groups alphabetically ordered, so we specify ascending order, or 0.

Again, we don't have a Key #3, so we specify Key #3 size, position and order as 0, 0, and 0 respectively.

The last lines close our input and two output files. The program prints us a reminder of the file names, then ends.

STREET.LST, the sorted version of all the streets contained in the file STREET.DAT, looks like this:

```
Achilles Drive           W
Cisco Kid Road           S
Darth Vader Road         W
Edward Teach Street      E
Ernest Hemingway Lane    W
Harold Hardrede Court    N
James Bond Court         S
Jason Bourne Road        N
John Carter Lane         N
John Silver Road         W
Kublai Khan Circle       S
Marco Polo Avenue        E
Michael Finn Drive       N
Mick Dundee Street       E
Robin Hood Avenue        S
Robinson Crusoe Drive    S
Sancho Panza Circle      E
Sherlock Holmes Avenue   E
Sinbad Street            N
William Tell Lane        W
```

The file ENSW.LST, which is the streets sorted according to their location relative to town center, then sorted alphabetically, looks like this:

```
Edward Teach Street      E
Marco Polo Avenue        E
Mick Dundee Street       E
Sancho Panza Circle      E
Sherlock Holmes Avenue   E
Harold Hardrede Court    N
Jason Bourne Road        N
John Carter Lane         N
Michael Finn Drive       N
Sinbad Street            N
Cisco Kid Road           S
James Bond Court         S
Kublai Khan Circle       S
Robin Hood Avenue        S
Robinson Crusoe Drive    S
Achilles Drive           W
Darth Vader Road         W
Ernest Hemingway Lane    W
John Silver Road         W
William Tell Lane        W
```

Remember, if you choose not to assign a third key, or perhaps even a second key, you still must place zeros in the size, position and order variables of the keys you omit.

Chapter 3 - COMMON Provides Common Variable Storage

This chapter discusses:

- What does COMMON do?
- How to load COMMON into memory
- How to use COMMON

WHAT IS COMMON?

COMMON is an external subroutine that allows you to place data into a common storage area of either user or system memory. The data can be numeric variables or string variables of up to 150 bytes (characters) in length. When this data is in user memory, it may be accessed by separate AlphaBASIC programs, as when chaining from one program to a second that requires variable information defined in the first program. When it is in user memory, the data is only common to programs run by the particular job that placed them in memory.

When in system memory, this common data can be used to pass messages between jobs, or for any other function that requires a data area accessible to more than one person.

The common data is placed in either user or system memory by an AlphaBASIC program. The idea is to assign a name to one or several packets of data, which can later, and at various times, be retrieved by other AlphaBASIC programs.

HOW TO LOAD COMMON INTO USER OR SYSTEM MEMORY

You ***must*** load the COMMON subroutine into memory before you use it from within an AlphaBASIC program.

You may load COMMON into either system or user memory. If you load COMMON into a user's memory partition, only that user can access the data stored by COMMON. If you load COMMON into system memory (making the data accessible to all users), be sure you assign a unique name for each packet of data.

To load COMMON into user memory, enter:

```
LOAD DSK0:COMMON.SBR[7,6] RETURN
```

After you see the AMOS prompt, you may run an AlphaBASIC program that uses the COMMON subroutine.

To load COMMON.SBR into system memory, you must have a line in your system initialization file that performs that function:

```
SYSTEM DSK0:COMMON.SBR[7,6]
```

For more information on loading subroutines into system memory during system boot-up, see your *System Operator's Guide*.

HOW TO USE COMMON

There are two things the AlphaBASIC program itself must do to use the COMMON subroutine. The program must define certain variables COMMON will use, and it must contain an XCALL command line using the name COMMON and certain parameter specifications.

Make sure you have loaded COMMON into system or user memory before using it.

Defining Variables

To use COMMON, you must first define certain binary variables that tell COMMON to send a packet to memory or to receive one from memory. If COMMON is going to receive data, you must also define a variable that will let you know if COMMON actually received the data packet it tried to get.

You define these binary variables by using MAP statements. MAP statements are discussed at length in your *AlphaBASIC User's Manual*. The MAP statements you see below will be sufficient for all but the most exotic programs using COMMON.

To send a packet of data to common memory, you must define a variable (we'll call it SEND), which must appear in the XCALL COMMON program line when you are sending, as:

```
MAP1  SEND,B,1,0
```

This one-byte binary variable always contains zero (the flag telling COMMON to send).

To receive a packet of data from common memory, you must define a two-byte binary variable (we'll call it RECEIVE) which must appear in the XCALL COMMON program line when you are receiving, to communicate two pieces of information to COMMON. The first byte must be a 1, which is the flag to COMMON you are going to receive a packet from memory (we'll name that byte F'RCV). The second byte (which we'll call RCVFLG) is a flag you can test after the XCALL COMMON subroutine is executed to see if you did in fact receive the packet. That two-byte binary variable is defined as:

```
MAP1  RECEIVE
      MAP2  F'RCV,B,1,1
      MAP2  RCVFLG,B,1,0
```

Again, F'RCV always contains a one (the flag telling COMMON to receive). RCVFLG functions as a flag to indicate whether or not COMMON finds the requested packet of information. If COMMON does not find that packet, it will return a zero in this byte; otherwise it is non-zero.

COMMON Format

The format is:

```
XCALL COMMON,send,"msgnam",variable(s)
```

or:

```
XCALL COMMON,receive,"msgnam",variable(s)
```

send	A one-byte binary variable that contains zero.
receive	A two-byte binary variable. The first byte must be set to one, and the second byte is a flag. If the flag is 0, COMMON did not find the packet; any non-zero number means it did. **Once you use COMMON to retrieve a data packet, the packet is erased from memory, and cannot be read again.**
"msgnam"	A string containing from one to six characters that specifies the name of the packet to be sent or received. String literals must be enclosed in quotation marks. String variables must be defined.
variable(s)	The variable to hold the data to be sent or received. The variable cannot hold more than 150 bytes.

If you load COMMON into system memory, make sure the MSGNAM is unique for each packet.

An Example Of COMMON

Let's create a pair of elementary AlphaBASIC programs and put a packet into user memory, then retrieve it. We assume that after you write and compile these programs, you will load the COMMON subroutine into user memory before running them, as we discussed above. To send a data packet to common memory, you may use a routine like this:

```
10  MAP1 SEND,B,1,0
20  MAP1 VAR,S,150
30  MAP1 PACKET,S,6
40  INPUT "Enter message (maximum of 150 characters): ",VAR
50  PRINT "Now enter name of data packet ";
60  INPUT "(up to 6 characters): ",PACKET
70  XCALL COMMON,SEND,PACKET,VAR
80  END
```

Line 10 defines the binary variable SEND as a zero. Line 20 defines the variable "VAR" as a string variable up to 150 characters in length (the maximum COMMON can handle). Line 30 defines a string variable called PACKET, which can be up to six characters in length.

Line 40 accepts a value from the terminal and assigns it to the variable VAR, which will make up the data in the packet you'll store in common memory. Line 60 accepts an input string that becomes the name of the packet.

Line 70 calls the COMMON subroutine, telling it to SEND to PACKET the variable information stored in VAR. When the program is run, the packet is placed in common memory.

To retrieve the packet under the name you input (defined as the string variable PACKET), you may use a routine like the following:

```
10  MAP1 RECEIVE
20       MAP2 F'RCV,B,1,1
30       MAP2 RCVFLG,B,1,0
40  MAP1 RETRIEVE,S,150
50  MAP1 PACKET,S,6
60  INPUT "Enter the name of the data packet: ",PACKET
70  XCALL COMMON,RECEIVE,PACKET,RETRIEVE
80  IF RCVFLG=0 PRINT "Message not found" &
    ELSE PRINT "The message is: ";RETRIEVE
90  END
```

Line 10 of the retrieving program is a level 1 MAP statement. The subsequent MAP2 statements pertain to it; when the variable RECEIVE is looked at, the associated information in lines 20 and 30 are automatically accepted as well.

Line 20 defines the binary variable F'RCV as a one, which later will tell COMMON to receive, rather than send. Line 30 contains RCVFLG, another binary byte. This one can be tested by the program following the XCALL to the COMMON subroutine. If this variable equals zero, the program knows that, for some reason, COMMON did not find the designated packet; a non-zero means it did find it.

Line 40 defines RETRIEVE as a string variable of up to 150 characters in length.

Line 50 defines PACKET as a string variable of up to six characters in length.

Line 60 asks the program user for the name of the packet in common memory that he or she wants to retrieve.

Line 70 calls the COMMON subroutine, which finds PACKET and gets the information contained there, which is put in the variable RETRIEVE.

Line 80 tests the binary flag to see if the packet was found and displays the appropriate message on your terminal. If the packet is found, its contents are displayed also. Then the program ends.

Sample runs of the two sample programs above look like:

```
RUN FIRST [RETURN]
Enter message (maximum of 150 characters): TEMPUS FUGIT! [RETURN]
Now enter name of data packet (up to 6 characters): MESAG1 [RETURN]
```

and:

```
RUN SECOND [RETURN]
Enter the name of the data packet: MESAG1 [RETURN]
The message is: TEMPUS FUGIT!
```

If you entered a message name that does not represent a packet in common memory, you would see the message from line 80 of the second program saying, "Message not found."

Chapter 4 - FLOCK Co-ordinates Multi-user File Access

This chapter discusses:

- What is FLOCK?
- What is a multiple update problem?
- What is an interconsistency problem?
- Loading and Using FLOCK
- What is deadlock, and how to prevent it

WHAT IS FLOCK?

The name FLOCK is an acronym for "File LOCKing." FLOCK helps prevent multiple update problems, interconsistency flaws, and other file access hazards that may occur if you are not the only user on your system. FLOCK provides a way to synchronize attempts at accessing files and devices so you and the other users can avoid partially updating or scrambling data.

The FLOCK routine has been replaced by the more efficient and easier-to-use automatic system file locking available on most AMOS systems. FLOCK is included here for compatibility purposes.

If you are worried about the type of problems that can occur when two or more people or programs access the same file at the same time, we recommend you use the AMOS file locking system, a system-wide, automatic file protection program that prevents file locking problems. FLOCK and XLOCK (described in the next chapter) are older programs effective only if all the programs involved use them. FLOCK can be useful if you need file-lock protection for only a few operations, or if your system is limited in memory and cannot support system file locking. As of AMOS releases 2.0 and later, AMOS automatically locks all files.

THE MULTIPLE UPDATE PROBLEM

This problem occurs when two programs are updating the same data record at about the same time. What can happen is the second program writes its data into the file over the data the first program writes, and the first program's data is lost.

WHAT IS AN INTERCONSISTENCY PROBLEM?

This problem is related to the multiple update problem. If a program is reading data from a file, and another program is changing those values at the same time, the data read from the file will be incorrect. Worse, some of the data may be correct, and some incorrect, depending on when the changes took place!

LOADING FLOCK INTO SYSTEM MEMORY

FLOCK resides in account DSK0:[7,6], and has a .SBR extension.

You ***must*** load FLOCK into system memory only; it will appear to work if you load it into user memory, but no file locking will actually occur.

To load FLOCK.SBR into system memory, you must have a line like this in your system initialization command file:

```
SYSTEM DSK0:FLOCK.SBR[7,6]
```

For more information on loading subroutines into system memory during system boot-up, see your *System Operator's Guide*.

FLOCK FORMAT

The format of FLOCK is:

```
XCALL FLOCK,action,mode,return-code,file,record
```

The ***return-code*** is a 6-byte floating point variable, and the other parameters are all either floating point expressions of positive integer values, or string expressions which represent positive integer values. They are described in the following sections.

Action and Mode

Action, modified by ***mode***, specifies the action for FLOCK to performed. A quick-reference summary of the actions and their modes is at the end of this chapter. The actions, and their modes, are:

Action 0, Mode 0 Requests permission to open ***file*** for non-exclusive use (that is, other users can access the file). The request is placed in a first-come-first-served queue and the program is delayed until the request can be granted.

Action 0, Mode 2 Requests permission to open ***file*** for exclusive use. The request is placed in a first-come-first-served queue and the program is delayed until the request can be granted.

Action 0, Mode 4 Requests permission to open ***file*** for non-exclusive use. If the request cannot be immediately granted, the ***return-code*** is 1.

Action 0, Mode 6 Requests permission to open ***file*** for exclusive use. If the request cannot be immediately granted, the ***return-code*** is 1.

Action 1, Mode 0 Informs FLOCK ***file*** has been closed. Unlocks the file. Implicitly informs FLOCK any processing of records in ***file*** has been completed (i.e., actions 5 or 6 are performed automatically as necessary).

Action 2, Mode 0 Informs FLOCK abnormal program termination is about to occur (e.g., during an error handling routine). Releases all locks on all files by performing action 1 as necessary.

Action 3, Mode 0 Requests permission to read ***record*** of ***file*** for non-exclusive use (i.e., record will not update the file). Permission to open ***file*** must already be granted. The request is placed in a first-come-first-served queue and the program is delayed until the request can be granted.

Action 3, Mode 2 Requests permission to read ***record*** of ***file*** for exclusive use (i.e., record will update the file). Permission to open ***file*** must already be granted. The request is placed in a first-come-first-served queue and the program is delayed until the request can be granted.

Action 3, Mode 4 Requests permission to read ***record*** of ***file*** for non-exclusive use (i.e., record won't update file). Permission to open ***file*** must already be granted. If request cannot be immediately granted, ***return-code*** is 1.

Action 3, Mode 6 Requests permission to read ***record*** of ***file*** for exclusive use (i.e., record will update file). Permission to open ***file*** must already be granted. If the request cannot be immediately granted, the ***return-code*** is 1.

Action 4, Mode 2 Requests permission to read/write all records of ***file*** for exclusive use (i.e., processing will update and possibly re-create the file). Permission to open ***file*** must already be granted. The request is placed in a first-come-first-served queue and the program is delayed until the request can be granted.

Action 4, Mode 6 Requests permission to read/write all records of ***file*** for exclusive use (i.e., processing will update and possibly re-create the file). Permission to open ***file*** must already be granted. If the request cannot be immediately granted, the ***return-code*** is 1.

Action 5, Mode 0 Informs FLOCK processing of ***record*** of ***file***, for which permission was granted by action 3, has been completed. The record is unlocked. If data has been buffered for output, it is written to the disk.

Action 6, Mode 0 Informs FLOCK exclusive processing of ***file***, for which permission was granted by action 4, has been completed. The file is unlocked. Any succeeding programs which are granted use of ***file*** by actions 3 or 4 will automatically re-open ***file***. This is done in case exclusive processing of ***file*** has caused it to be re-created. If data has been buffered for output, it is written to the disk.

File

File specifies a file-channel number. File is ignored by action 2 and may be omitted if ***record*** is also omitted. The file specified may be either random or sequential for actions 0 and 1, but must be a random file for all other actions.

For FLOCK to function properly, file-channel numbers should denote specific and unique files. This means you must systematically assign file-channel numbers to your files when designing applications programs, being careful to assign the same numbers to the same files and different numbers to different files. File-channel numbers 1 through 999 have been reserved for use by Alpha Micro software. Although there is nothing to prevent your programs from using these numbers, we advise you not to do so in conjunction with FLOCK so no conflict can arise between your application programs and any present or future Alpha Micro software on your system.

Record

Record specifies a logical record number. For actions 0, 1, 2, 4, and 6, record is ignored and may be omitted.

Return-code

Return-code denotes a variable in which FLOCK places a number indicating the success or failure of an action:

CODE	MEANING	FOR ACTIONS
0	Successful	(All)
1	Resource unavailable	(0, 3, 4)
2	Open request already granted	(0)
3	Permission to open must first be granted	(1, 3 - 6)
4	Duplicate request for use of some record in the file	(3 and 4)
6	Permission to use record must first be granted	(5 and 6)
100	Unimplemented Action	
101	File-channel # not open for random processing	(3 - 6)
102	File-channel already open for ISAM indexed file	(0)
103	Less than 15 queue blocks are available	(0, 3, 4)
104	Illegal record number	(3 - 6)

A return-code greater than 1 indicates some programming error. For calls to FLOCK which do not use modes 4 or 6, you should use the following statement while debugging your program:

```
IF Return-Code > 1 THEN PRINT “FLOCK Error” : STOP
```

For calls which use modes 4 or 6, return-code = 1 should be checked to determine if FLOCK was able to immediately satisfy the request. Modes 4 and 6 are generally used in this way to allow the user to cancel a request which may involve a lengthy delay.

QUEUE BLOCK REQUIREMENTS

The FLOCK subroutine builds its dynamic tables out of monitor queue blocks. The monitor queue is a list of blocks of system memory which are linked to each other in a forward chain.

Each queue block is currently eight words (16 bytes) in size (this value may change with a later release of the file system). During normal monitor operations, various functions use these queue blocks to perform certain tasks.

Before running any AlphaBASIC program using FLOCK or XLOCK, be sure the monitor is configured to make an adequate number of queue blocks available.

The monitor is initially generated with 20 free blocks in the available queue. You may modify the system initialization command file to allocate more queue blocks by adding the `QUEUE nnn` command anywhere in the system initialization command file before the final SYSTEM command.

When the QUEUE nnn command is executed, ***nnn*** more queue blocks will be allocated for general use. For more information on modifying the system initialization command file, see your *System Operator's Guide*.

You may enter QUEUE [RETURN] at AMOS level to see your system's use of queue blocks. The system will display the current number of free queue blocks in the available queue list.

The number of queue blocks FLOCK uses varies with the number of jobs accessing files, the number of files open at one time, and the number of records open for each file. Currently, at any given moment during the use of FLOCK, the number of queue blocks being used equals:

Twice the number of different files open using FLOCK
+
The number of different records open using FLOCK
+
The number of jobs with files open using FLOCK
+
The TOTAL number of unclosed FLOCK opens (i.e., # of action 0s)
+
The TOTAL number of unreleased record uses (i.e., # of action 3s)

The last two factors of this equation anticipate circumstances where the same file and/or the same record is being accessed by more than one job at a time. If two jobs are reading the same file, that is two opens or two action 0s.

HOW TO USE FLOCK

There are three levels of increasing complexity with which FLOCK subroutine calls may be incorporated into a program system:

1. Use actions 0 through 2 to implement file-open interlocks.
2. Use actions 0 through 2 to implement file-open interlocks and use actions 3 and 5 to implement individual record-update interlocks.
3. Use actions 0 through 2, 4, and 6 to implement complete file interlocks and use actions 3 and 5 to implement individual record-processing interlocks.

In your design you are free to trade off complexity for performance, so long as you use a single level of complexity consistently for any given data file.

File-Open Interlocks

Using just actions 0 through 2, it is possible to make a very simple file access coordination scheme which solves the problems of multiple updates and interconsistency.

Action 0, mode 0 or 4, is used before opening a file for input only (that is, opening a file for random processing, upon which only READs will be performed).

Action 0, mode 2 or 6, is used before opening a file for output (e.g., a file open for random processing, upon which READs or WRITEs will be performed, or a file which may be re-created).

Finally, action 1 is used after closing any file, and action 2 is used before any abnormal termination points in the program.

Record-Update Interlocks

Most programs open files when the programs begin, and close them when they end. The programs may not actually need the files to be open throughout execution, but by not repeatedly opening and closing the files, the programs avoid many undesirable delays.

File-open interlocks lock out the entire file; if a file is open throughout the run of a program, and thus unavailable to programs run by other users, serious or annoying delays can result.

Although file-open interlocks do prevent concurrency problems, they generally reduce concurrency far more than is necessary. Typically, file-open interlocks lock out the entire file to prevent access to the single record. Locking out an entire file to prevent access to a single record is like using a sledge hammer to drive a push-pin. All that is actually necessary is to delay any other user attempting to modify the record until the user originally accessing the record is done.

Consider an application in which you and several other users interactive update an employee record file. Assume files are kept open only where required. Once you display an employee's record, all the other users must wait for you to finish making changes to that record before they can, in turn, access it; otherwise two users might concurrently attempt to update the same employee record.

This could result in a multiple update problem. In other words, all other users must wait for one user to enter changes to the employee's record before any other user can access and modify that record.

This is called a record-update interlock, and is a less severe restriction to all the users accessing a file than a file-open interlock is. Remember, when doing a record-update interlock, FLOCK converts logical record numbers into physical block numbers. All record locking operations are done on physical blocks, not logical records. If both you and another user attempt to lock two separate logical records within the same physical block, you will see the error message: Record already locked.

Actions 3 and 5 of FLOCK permit control of concurrent access to individual records. Action 3, mode 0 or 4, is used before reading a sequence of records which will not be used for updating, in order to prevent interconsistency errors.

Action 5 is used after the sequence of reads. Action 3, mode 2 or 6, is used before reading records which will be used for updating. Action 5 is used again after rewriting the records.

Improved File Interlocks

We said file-open interlocks can make long delays for any users trying to access a file after one user has opened it and therefore locked them out. Nevertheless, it is sometimes necessary to lock an entire file for exclusive use.

For example, if file XYZ is becoming full, you might wish to copy the file XYZ into a new, larger file TEMP, and then delete XYZ and rename TEMP to XYZ. Or, as another example, you might wish to reorganize an index and data file. Obviously, during these maneuvers, you want assurance no other user can access the file.

Action 4 obtains exclusive access to a file by obtaining exclusive access to all the records of that file. Exclusive access is relinquished by using action 6. Action 3, mode 0 or 4, is necessary before reading a sequence of records in order to avoid the interconsistency problem. If action 4 is used, it is necessary to use action 3, mode 0 or 4, before reading individual records which won't be used for updating. This is because a user who has exclusive use of a file can re-create it, which requires all other users with the file open must then reopen it. Action 3 performs the necessary re-openings.

DEADLOCK, AND HOW TO PREVENT IT

For the purposes of the following discussion, having permission to open a file or use a record is referred to as "possessing a resource."

The possession of a resource by some job XYZ can directly or indirectly cause the execution of other jobs to be delayed. It is then possible for one of these delayed jobs to possess a resource needed by job XYZ, thus causing execution of job XYZ to be delayed also.

This is known as a DEADLOCK. None of the jobs involved can proceed since each requires a resource owned by one of the other jobs involved. The situation is permanent because none of the jobs involved can proceed until one of the other jobs proceeds and relinquishes a needed resource.

DEADLOCK can only occur if a job requests more than one resource simultaneously. There is a simple way to prevent DEADLOCK, a method which, in most cases, is feasible to implement. The method is: ALWAYS request resources in the same order. Here is a simple illustration of the principle. First we consider what can happen if resources are requested in differing order in two programs:

```
XCALL FLOCK,0,2,RET,1001                    !  PROGRAM 1
XCALL FLOCK,0,2,RET,1002
!                                   ** BODY OF PROGRAM **
XCALL FLOCK,1,0,RET,1002
XCALL FLOCK,1,0,RET,1001
END

XCALL FLOCK,0,2,RET,1002                    !  PROGRAM 2
XCALL FLOCK,0,2,RET,1001
!                                   ** BODY OF PROGRAM **
XCALL FLOCK,0,2,RET,1001
XCALL FLOCK,0,2,RET,1002
END
```

Consider the following sequence of execution:

1. Program 1 gets exclusive permission to open file 1001.
2. Program 2 gets exclusive permission to open file 1002. It then tries to access file 1001, and must be delayed because Program 1 already has exclusive permission to open file 1001.
3. Program 1 tries to access file 1002, and must be delayed because Program 2 already has exclusive permission to open file 1002.

At this point, programs 1 and 2 have both been delayed. Since no other programs are present, the reasons for their delays will remain unchanged. DEADLOCK has occurred.

But DEADLOCK will not occur if program 2 requests permission to open files 1001 and 1002 for exclusive use in the same order as program 1. For DEADLOCK to occur, program 1 must be granted permission to open file 1001 for exclusive use, but be delayed permission to open file 1002 for exclusive use.

However, if program 1 is granted permission to open file 1001 for exclusive use, the corrected program 2 (a duplicate of program 1) will not be allowed to execute further; thus it will be unable to obtain permission to open file 1002 for exclusive use. DEADLOCK cannot occur.

QUICK REFERENCE SUMMARY OF ACTIONS/MODES

ACTION 0: Request to open file.

MODE 0: Non-exclusive; delay until free
MODE 2: Exclusive; delay until free
MODE 4: Non-exclusive; return-code = 1 if not free
MODE 6: Exclusive; return-code = 1 if not free

ACTION 1: Tells FLOCK the file is closed. Releases the lock. Actions 5 and 6 performed as necessary.

ACTION 2: Tells FLOCK a program abort is about to occur. Releases all locks on all files by using action 1 as necessary.

ACTION 3: Request to read record.

MODE 0: Non-exclusive; delay if not free. Action 0 must already have been granted.
MODE 2: Exclusive; delay if not free. Action 0 must already have been granted.
MODE 4: Non-exclusive; return-code = 1 if not free. Action 0 must already have been granted.
MODE 6: Exclusive; return-code = 1 if not free. Action 0 must already have been granted.

ACTION 4: REQUEST TO READ/WRITE ALL RECORDS.

MODE 2: Exclusive; delay if not free. Action 0 must already have been granted.
MODE 6: Exclusive; return-code = 1 if not free. Action 0 must already have been granted.

ACTION 5: Tells FLOCK you have finished processing the record requested by a previous action 3 call. Any buffered data is output to the disk.

ACTION 6: Tells FLOCK you have finished processing the file requested by a previous action 4 call. Any buffered data is output to the disk.

Chapter 5 - XLOCK
Sets and Tests Device and File Locks

This chapter discusses:

- What is XLOCK?
- Loading XLOCK into system memory
- The XLOCK format
- How to use XLOCK

WHAT IS XLOCK?

XLOCK is an external subroutine your AlphaBASIC program can call to set and test device and file "locks."

A lock is a tool to help you synchronize attempts to access devices, files, and other resources. You can imagine the problems that result when you have two users trying to update the same record of the same file at the same time.

A lock is an entity created by a program to help it keep track of whether a certain device, file, etc., is in use at the specific time the program wants to access it. The general way the locking system works is this:

- When you want to prevent access to a file or a device while your program accesses it, you set a system lock on it.
- Whenever you want to access a device or file, your program tries to set the lock associated with that item; if it is already set, you know another user's program is using the device or file.
- When you are finished accessing a device or file, you clear the lock so other programs can access the device or file.

Note that a system lock is NOT a security device—it's a convenience. If a program wants to allow its users to write to a file without checking to see if another user is there first, it can do so (and run the risk of creating chaos).

For file locking purposes, the XLOCK routine has been replaced by the more efficient and easier-to-use automatic system file locking available on most AMOS systems. XLOCK is included here for compatibility purposes.

An XLOCK system lock simply provides a convenient way to help a program keep its users from conflicting in their attempts to use system resources.

The only job that can clear a lock is the job that originally set the lock. AlphaBASIC does not automatically clear locks when a program exits, so be careful that your program clears any locks it has set before it exits. For more background information on why locks are necessary, see Chapter 4.

LOADING XLOCK INTO SYSTEM MEMORY

Before you use any program that uses XLOCK, remember to allocate enough system queue blocks for those XLOCKs. See the section in the FLOCK chapter under Queue Block Requirements.

You MUST include the file DSK0:XLOCK.SBR[7,6] in system memory before you can use an AlphaBASIC program that calls XLOCK.

To load XLOCK.SBR into system memory, you must have a line in your system initialization command file that performs that function:

```
SYSTEM BAS:XLOCK.SBR
```

For more information on loading subroutines into system memory during system boot-up, see your *System Operator's Guide to the System Initialization Command File*.

THE XLOCK FORMAT

The format for XLOCK is:

```
XCALL XLOCK, mode, lock1, lock2
```

Mode is the function you want to perform. The modes are:

0	Set lock and return.
1	Set lock (Wait if locked; then set).
2	Clear lock (if set by your job).
3	Return a list of all system locks and the jobs that set them.

Each mode is discussed in more detail below. ***Lock1*** and ***lock2*** specify the first and second digits of the lock code (see below).

Use MAP statements at the front of your program to define mode, lock1, and lock2 as two-byte BINARY variables (they may not be floating point or string variables). For example:

```
MAP1  MODE,B,2
MAP1  LOCK1,B,2
MAP1  LOCK2,B,2
```

Before you call XLOCK, your AlphaBASIC program must first set up the correct values for mode, lock1, and lock2.

XLOCK parameters must be defined on even-byte boundaries in memory (that is, the variables must begin on word boundaries). Variable structures defined at a MAP1 level always begin on a word boundary. Therefore, the easiest way to be sure XLOCK arguments begin on a word boundary is to define them in MAP1 statements (as in the example above). If you do define XLOCK parameters in deeper level MAP statements (e.g., MAP2 or MAP3), make sure the variables begin on a word boundary by keeping the number of bytes defined an even number. For example, this definition:

```
MAP1  PARAMETERS
      MAP2  FILL,S,1
      MAP2  L1,B,2
      MAP2  L2,B,2
      MAP2  MODE,B,2
```

will cause XLOCK to fail; however, removing the definition for FILL (which pushed the XLOCK parameters onto an odd-byte boundary) will correct the problem.

THE LOCKS

A system lock is a two-level numeric lock; the number representing either level may be from 1 to 65535. A value of zero in either position acts as a wildcard. That is, any number will match in that position when it comes to clearing or setting that lock. Some typical locks are:

```
1,1
1,2
4,0
100,100
```

The numbers you choose are up to you. You may choose to assign some meaning to the numbers (for example, the first number might be the file-channel number of the file you want to lock, and the second number might be the number of the record within that file you want to lock).

Since both numbers in the lock may range from 1 to 65535, the actual possible number of unique locks is 65535 * 65535 = 4,294,836,225. However, every time you create a lock, the system sets aside a block in the monitor queue in system memory for that lock, which is not returned to the available list until the lock is released by the job that has it locked.

Since there are initially only 20 queue blocks available, it’s a good idea to keep the number of locks to a minimum. A good rule is a program should not have more than two or three locks active at any one time. As you clear a lock, that queue block becomes available again.

THE MODES

The mode argument in the XLOCK call line can contain one of four values (0-3) which selects one of the four possible locking modes:

Mode 0 (Lock and Return)

This mode tells XLOCK to create a lock with the value ***lock1,lock2***. If the lock already exists (i.e., some other job is accessing the file or device you want to use), XLOCK returns with ***mode*** equal to the number of the job that set the lock. A job number is assigned to each job in the order the jobs were defined in the JOBALC command in the system initialization command file. For example, the first job defined in the JOBALC command line is Job #1. The SYSTAT command lists the jobs in this order.

If the lock does not already exist, XLOCK creates it and returns with a zero in ***mode***. You've now set the lock.

Mode 1 (Lock and Wait)

This XLOCK mode is identical to mode 0, except that if the lock already exists, XLOCK tells the system to put your job to sleep until the lock is cleared. That means your job will be in an inactive state (except for waking at every clock tick to test the status of the lock) until the job that originally set the lock clears it.

If you use this mode, take into consideration the fact that another user may be waiting for the same lock; it's possible the lock might be cleared and then grabbed up either by the same or another job before your job wakes up.

Mode 2 (Clear Lock)

XLOCK clears the lock specified by ***lock1*** and ***lock2*** and returns to your program. A zero returned in ***mode*** indicates the lock you tried to clear wasn't set by your job; a one returned indicates you successfully cleared one lock; a number greater than one indicates you cleared more than one lock (which means lock1 or lock2 was originally set to zero—the wildcard value). You may never use XLOCK to clear a lock not set by your job.

If you attach your terminal to another job, XLOCK considers you a new job.

Mode 3 (List Locks)

Mode 3 returns a complete list of all the locks set on the system and the numbers of the jobs that set them. When you use mode 3, ***lock2*** must represent a mapped array large enough to hold the expected data. When XLOCK returns from a mode 3 call, ***mode*** contains the number of locks set on the system, ***lock1*** contains your job number, and ***lock2*** contains one three-word entry for each lock set on the system (you must set up this entry as three binary words in a MAP statement). The first two bytes hold the job number; the second and third words hold the actual lock1 and lock2 values of the specified lock. The following is an example of how to set up the MAP statement for a mode 3 call:

```
MAP1  MODE,B,2
MAP1  MYJOB,B,2
MAP1  LIST'ARRAY
      MAP2  LOCKENTRY(25)
            MAP3  JOBNUMBER,B,2
            MAP3  L1,B,2
            MAP3  L2,B,2
            .
! Start of the program here
            .
MODE = 3
XCALL XLOCK,MODE,MYJOB,LIST'ARRAY
            .
! Rest of the program here
```

WILDCARDS

A system lock consists of two numbers, the values of lock1 and lock2. If either of these two numbers is a zero, that number is a wildcard and any number between 1 and 65535 will match it. A wildcard is a symbol that is matched by any other symbol.

You can use wildcards for various purposes. For example, suppose you decide the lock1 value is going to represent a particular file and the lock2 value will represent a particular record in that file. If you want to stop all references to that file while your program is accessing it, you would set the lock with a zero in lock2 and the number representing your file in lock1.

Anyone who tries to set a lock with the same lock1 value as your lock won't be able to do so; the system will tell that program that lock already exists (since your wildcard in lock2 will match any number it may try in that position). No one (including yourself) will be able to set a lock with the same lock1 value until you clear the lock.

Note that setting a lock with both numbers zero will prevent anyone from setting a lock, since the system will report all possible locks are already set.

PROGRAMMING EXAMPLES

The following is a demonstration program you may want to use to experiment with XLOCK, and to get a feeling for how it works. It asks you for the values of ***mode, lock1***, and ***lock2***, and then reports back on the results of the locking operation you asked for.

```
!  Sample Program to Illustrate File Locking
MAP1  FLAG,F
MAP1  COUNTER,F
MAP1  MODE,B,2
MAP1  LOCK1,B,2
MAP1  LOCK2,B,2
MAP1  LOCK'ARRAY
      MAP2  LOCK'ENTRY(25)
            MAP3  JOB,B,2
            MAP3  L1,B,2
            MAP3  L2,B,2
```

```
START:
  INPUT "MODE,LOCK1,LOCK2: ",MODE,LOCK1,LOCK2
  FLAG = MODE
  IF MODE = 3 GOTO DISPLAY
  XCALL XLOCK,MODE,LOCK1,LOCK2
  PRINT "Mode = "; MODE
  IF FLAG = 0 AND MODE <> 0 PRINT "Lock already set."
  IF FLAG = 2 AND MODE  = 0 PRINT "You didn't set that lock."
  IF FLAG = 2 AND MODE  = 1 PRINT "You cleared the lock."
  IF FLAG = 2 AND MODE  > 1 THEN &
     PRINT "You cleared more than one lock."
  GOTO START
DISPLAY:
  XCALL XLOCK, MODE, LOCK1, LOCK'ARRAY
  PRINT "Your job number is: ";LOCK1
  PRINT "Current locks in use = ";MODE
  IF MODE = 0 GOTO LOOP
  FOR COUNTER = 1 TO MODE
      PRINT SPACE(5);
      PRINT STR(L1(COUNTER)) + "," + STR(L2(COUNTER));
      PRINT SPACE(4); : PRINT "(Job";JOB(COUNTER);")"
  NEXT
LOOP:
  PRINT : GOTO START
```

XLOCK is often used to lock individual records within a file so more than one user can update that file at the same time. Lock1 might contain a number representing the particular file you want to open for multi-user updating (perhaps by containing the file's file-channel number). Lock2 might hold a number representing the specific record you want to update within the file.

Calculating Record Numbers

We assume you will usually be using XLOCK to control multi-user updating of random files. For information on random files, see your *AlphaBASIC User's Manual*. If you are going to be locking a specific file record, you need to understand the relationship between disk blocks and file records. A record (sometimes called a "logical record") is a grouping of data you define; you also define the length of that record.

Just as an example, let's define a file record that contains 6 bytes for a customer ID number, 24 bytes for a customer name, 10 bytes for the name of the customer's sales contact, and 10 bytes for the customer phone number.

This file record would then contain 50 bytes. A disk block is a physical grouping of data on the disk that is always 512 bytes long. The monitor always transfers disk information in this 512-byte block.

AlphaBASIC unblocks a disk block into smaller groups—your logical records. For example, one disk block (512 bytes) would contain 10 of the logical records we defined above (50 * 10 = 500) with 12 bytes left over. No logical record is ever larger than a disk block. You specify the size of your logical record in the OPEN statement for the file.

The reason for our explanation above is this: if you want the ***lock2*** value to contain the number of the record you are updating, it must contain the relative number of the DISK BLOCK being used, and not the logical record number.

When AlphaBASIC unblocks a disk block into logical records, it brings the entire disk block into your memory partition. Even if you are only updating one logical record in that disk block, the entire disk block remains in your memory area until you either close the file or read a logical record in a different disk block.

What this means is that more than one user could try to write out the same disk block at the same time even though they are updating different logical records. So, you must prevent access, not only to the logical record you are updating, but to the entire disk block that contains it.

You must calculate the relative disk block number yourself by dividing the logical record number by the blocking factor (the blocking factor is the number of logical records that can fit in one disk block). In the example above where we had logical records 50 bytes long, the blocking factor is 10. Remember that each disk block is 512 bytes long and will be blocked to contain as many logical records as will fit.

If one of your lock digits is the disk block number, you can prevent access to the entire disk block; no one can access any of the logical records in the disk block until you clear the lock.

The lock wildcard symbol is a zero, so calculate your disk blocks beginning with one instead of zero. Before you unlock the lock on a disk block, force the system to write that record by reading a logical record that falls outside of that disk block.

You may also use the RANDOM'FORCED mode in your OPEN statement to force AlphaBASIC to perform a disk read or a disk write every time you access the file. See your *AlphaBASIC User's Manual* for more information.

The sample program below may help to clarify the last few paragraphs.

Sample Program to Illustrate File Record Locking

```
! Sample Program to Illustrate File Record Locking
! Remember to load XLOCK.SBR before running!
MAP1  MODE,B,2                    ! Define locking variables
MAP1  LOCK1,B,2
MAP1  LOCK2,B,2
MAP1  LOGICAL'RECORD              ! Define logical record
      MAP2  CUST'ID,F,6           ! contents - 50 bytes
      MAP2  CUSTOMER,S,24       ! of customer info.
      MAP2  CONTACT,S,10          ! Customer ID is actually
      MAP2  PHONE,S,10            ! logical record number.
MAP1  RECORD'SIZE,F,,50
! Scratch variables:
MAP1  RECORDNUM,F                 ! Logical record number
MAP1  FLAG,F
MAP1  QUERY,S,1
   ! Begin program:
```

```
START:
  LOOKUP "CUSTID.DAT",FLAG        ! If file doesn't exist,
  IF FLAG = 0 THEN GOTO FILE'ERR  !   report error and exit.
  OPEN #100, "CUSTID.DAT",RANDOM,RECORD'SIZE,RECORDNUM
  LOCK1 = 100                     ! 100 represents CUSTID file
  PRINT "Welcome to the Customer Maintenance Program." : PRINT
LOOK:
  PRINT "Please enter customer identification ";
  INPUT "number: ",RECORDNUM
    ! Note: Customer ID is just number of that logical record.
    ! Calculate relative disk block number (assumes logical
    ! records begin with zero):
  LOCK2 = INT(RECORDNUM/10) + 1
    ! Lock the disk block used by the record.
  XCALL XLOCK,MODE,LOCK1,LOCK2
  READ #100, LOGICAL'RECORD
  PRINT "Customer information:"
  PRINT TAB(5);"Customer ID#: ";CUST'ID
  PRINT TAB(5);"Customer name: ";CUSTOMER
  PRINT TAB(5);"Sales contact: ";CONTACT
  PRINT TAB(5);"Phone #: ";PHONE

UPDATE:
  INPUT "Do you wish to change any info? ";QUERY
  IF UCS(QUERY) = "N" THEN GOTO LOOP
  PRINT "Customer ID: ",CUST'ID
  INPUT "Enter customer name: ";CUSTOMER
  INPUT "Enter sales contact: ";CONTACT
  INPUT "Enter phone number: ";PHONE
  WRITE #100, LOGICAL'RECORD
    ! Force BASIC to bring different disk block into memory.
    ! (If we are in first disk block, since blocking factor is
    ! 10, record number >= 10 will force in next disk block)
  IF LOCK2 = 1 THEN RECORDNUM = 10 ELSE RECORDNUM = 0
    ! Now bring in different disk block:
  READ #100,LOGICAL'RECORD
    ! Release the lock.
  MODE = 2
  XCALL XLOCK,MODE,LOCK1,LOCK2
LOOP:
  INPUT "Do you wish to see info on another customer? ",QUERY
  IF UCS(QUERY) = "Y" THEN GOTO LOOK
EXIT:
  PRINT "Returning you to Command Level..."
  CLOSE #100
  END
FILE'ERR:                             ! File did not exist.
  PRINT "File error.  Please see System Operator."
       END
```

Chapter 6 - SPOOL Prints Files

This chapter discusses:

- What is SPOOL?
- The SPOOL format
- How do you use SPOOL?

WHAT IS SPOOL?

The SPOOL call is used to send disk files to a printer. "SPOOL" is actually an acronym meaning "Simultaneous Printer Output On-line." To "spool" a file is to insert it into the printer queue, after which you can continue to do other things while your file waits in the queue for its turn to be printed.

There are many options you can use to control just how and where your files are printed.

You don't have to load the SPOOL subroutine into system or user memory in order to access it from an AlphaBASIC program, but you may want to in order to increase execution speed.

As of AMOS/L release 1.3, two different print spoolers are available—the old line print spooler and the Task Manager print spooler. The SPOOL subroutine will work with either spooler. The Task Manager print spooler replaces the line print spooler, so you should convert any programs to the Task Manager format.

There are two formats for the SPOOL XCALL. The traditional XCALL is provided for compatibility purposes. The expanded SPOOL XCALL allows you to take advantage of all the new features of the latest spooler.

TRADITIONAL SPOOL FORMAT

The traditional SPOOL call has the format:

```
XCALL SPOOL,file${{,printer$}{{,switches}{{,copies}
           {{,form$}{{,width}{{,lpp}
```

file$ A string variable or expression giving the specification of the file you want to print. If you specify a file which does not exist, SPOOL doesn't tell you it can't find the file (but of course it doesn't print anything).

printer$ A string variable or expression giving the name of the printer you want to send the file to. If printer is omitted or is a null string, SPOOL uses the default printer. If you want to use the default printer and also wish to use one or more of subsequent features (switches, copies, etc.), place a null string (“”) in the printer position of the program line (e.g., `XCALL SPOOL,"DATA.TXT","",5`).

switches A floating point variable or expression specifying various control switches and flags that affect the printing of the file. The switches SPOOL uses are the same as the switches of the same names used by the monitor PRINT or PRNT commands (see your *System Commands Reference Manual*.

Each switch you can use has a numeric code associated with it. Set control switches by putting the sum of the appropriate switch codes into the switches variable. For example, if you want to use the BANNER and DELETE switches, load ***switches*** with 5. The switch codes are:

```
BANNER      1
NOBANNER    2
DELETE      4
NODELETE    8
HEADER      16
NOHEADER    32
FF          64
NOFF        128
WAIT        256
INFORM      512
NOINFORM   1024
KILL       2048
```

copies A floating point variable or expression that specifies the number of copies to be printed. If you omit copies or it is zero, the line printer spooler program prints one copy.

form$ A string variable or expression that specifies the form on which the file is to be printed. If you omit form or it is a null string, (“”), the spooler uses NORMAL form.

width A floating point variable or expression giving the width (in characters) of the page. SPOOL only uses this if you have used the HEADER option. It does NOT affect the number of characters in the print line—only the text in the banner and header.

lpp A floating point variable or expression giving the number of lines per page. SPOOL only uses this if you used the HEADER option.

TASK MANAGER SPOOL FORMAT

The new SPOOL format is:

```
XCALL SPOOL,file$,spool'table
```

Spool'table is a mapped variable with the form:

```
MAP1  SPOOL'TABLE
      MAP2  PRINTER,S,6,""        ! Printer specification
      MAP2  CPU,F,6,0             ! CpuID for Network access
      MAP2  SWITCHES,F,6,0        ! Switches (see above)
      MAP2  COPIES,F,6,2          ! # of copies to print
      MAP2  BANNER,S,50,""        ! Banner before file
      MAP2  LPP,F,6,0             ! Lines per page
      MAP2  WIDTH,F,6,0           ! Width of page
      MAP2  FORMS,S,6,""          ! Form to print on
      MAP2  PRI,F,6,0             ! Queue priority of file
      MAP2  AFTER'DATE,B,4,0      ! Print after date
      MAP2  SEP'DATE,@AFTER'DATE
            MAP3  MONTH,B,1
            MAP3  DAY,B,1
            MAP3  YEAR,B,1
      MAP2  AFTER'TIME,B,4,0      ! Print after time
      MAP2  SEP'TIME,@AFTER'TIME
            MAP3  HOURS,B,1
            MAP3  MINUTES,B,1
            MAP3  SECONDS,B,1
      MAP2  RESTART,F,6,0         ! Restart yes or no
      MAP2  START,F,6,0           ! Start at page
      MAP2  FINISH,F,6,0          ! End at page
      MAP2  LIMIT,F,6,0           ! Limit of pages to print
```

See the PRNT reference sheet in your *System Commands Reference Manual* for more information on the various options.

HOW TO USE SPOOL

Having defined the MAP structure above, define those fields you need, and then call the subroutine. For example:

```
SWITCHES = 512
COPIES = 2
XCALL SPOOL,FILE$,SPOOL'TABLE
```

Chapter 7 - XMOUNT Mounts a Disk

This chapter discusses:

- What does XMOUNT do?
- The format of XMOUNT
- How to use XMOUNT to mount a disk

WHAT IS XMOUNT?

XMOUNT allows you to mount a disk from within an AlphaBASIC program without leaving AlphaBASIC. This works with systems that have floppy disk drives or systems that contain removable hard disk packs. You should call XMOUNT whenever you want to change one of these types of disk and your AlphaBASIC program is going to sort files or create new files on the newly changed disk.

You must always mount a disk after you've physically placed it in the drive, and before you write to it; otherwise the system will think the old disk is still in the drive. When it comes time to write information out to the new disk, the disk's bitmap will be wrong, and the system will try to write to the new disk as if it had the same areas free as the old one. Besides bringing into memory the proper bitmap, XMOUNT also loads in the alternate track table, if any, for the specified device.

NEVER mount or unmount a disk while someone is accessing that disk. Doing so will corrupt the data on the disk.

It is not necessary to load the XMOUNT subroutine into system or user memory. However, the XMOUNT subroutine is fully re-entrant, so for increased access speed you may load it into system memory using the SYSTEM command in your system initialization command file. See your *System Operator's Guide to the System Initialization Command File* for information on the system initialization command file.

XMOUNT FORMAT

The format of XMOUNT is:

```
XCALL XMOUNT,device{/U},volume'id
```

device A string variable or expression that represents a device specification (e.g., "DSK1:"). You may optionally follow the device specification with ***/U*** to unmount the device.

volume'id A string variable in which the volume ID of the mounted device will be returned. This variable must be 10 bytes long. If it is not specified, the label block will not be read. This variable is ignored if the /U option is used.

If you specify the unmount option, the "U" must be uppercase. When you unmount a disk, you prevent AlphaBASIC and most system programs from accessing that device.

HOW TO USE XMOUNT

The XMOUNT subroutine requires a string variable or expression to represent the specification of the device to be mounted (or unmounted), which is separated by a comma from the word XMOUNT. For example:

```
XCALL XMOUNT,"HWK1:"
```

or:

```
MAP1  DEVICE,S,9
DEVICE = "HWK1:"
XCALL XMOUNT,DEVICE
```

You may similarly unmount a disk by making the switch part of the same expression or string variable:

```
XCALL XMOUNT,"HWK1:/U"
```

or:

```
MAP1  DEVICE,S,9
DEVICE = "HWK1:/U"
XCALL XMOUNT,DEVICE
```

The only option available when using XMOUNT (other than the switch to unmount a disk) is the ability to store the volume ID of the newly mounted disk within a string variable, perhaps to be displayed immediately after using the XMOUNT subroutine so the program user is sure he or she put the right disk in the drive.

XMOUNT recognizes this option when it sees a string variable following the device specification string or expression (and separated from it by a comma). XMOUNT returns the volume ID of the disk as that variable, which then may be displayed or tested. For example:

```
MAP1  VOLUME'ID,S,10

MOUNTING:
     XCALL XMOUNT,"HWK1:",VOLUME'ID
     PRINT VOLUME'ID;" is mounted."
     IF VOLUME'ID <> "ARCHIVE" &
        THEN GOTO WRONG'DISK
     GOTO CONTINUING

WRONG'DISK:
     PRINT "This is not the ARCHIVE disk."
     PRINT "You may abort the program or place the correct"
     PRINT "disk in the drive.  To abort use Control-C."
     STOP
     GOTO MOUNTING

     CONTINUING:
           .
           .
```

If the volume ID string variable is omitted or is too small, or if a follows the device specification string variable or expression, the volume ID variable is ignored and returns a null (empty) string.

Chapter 8 - GTLANG Returns Language Definition Table

This chapter discusses:

- What is GTLANG?
- The format of GTLANG
- How to use GTLANG

WHAT IS GTLANG?

GTLANG allows you to access the contents of the language definition table currently defined for your job. It returns many different parameters which are language dependent. These parameters can be used by your program to adapt the program to the currently selected language. Additional information about languages and their use can be found in your *System Operator's Guide*. It is not necessary to load GTLANG into system or user memory, but, since it is fully re-entrant and re-usable, you may do so for increased speed.

GTLANG FORMAT

```
XCALL GTLANG,status,GTLANG'MAP
```

Status is a floating point variable in which the success (0) or failure (-1) of the operation is returned.

GTLANG'MAP is a MAP1 level area which will receive the descriptive information about the job's language. It must be defined like this:

```
MAP1  GTLANG'MAP
      MAP2  LANG'NAME1,S,20
      MAP2  LANG'NAME2,S,20
      MAP2  LANG'EXTENSION,S,4
      MAP2  LANG'CURRENCY'SYM,S,4
      MAP2  LANG'CURRENCY'POS,F,6
      MAP2  LANG'CURRENCY'SPC F,6
      MAP2  LANG'CURRENCY'SIZE,F,6
      MAP2  LANG'THOUSANDS,S,1
      MAP2  LANG'DECIMAL,S,1
      MAP2  LANG'DATE'FORMAT,F,6
      MAP2  LANG'TIME'FORMAT,F,6
      MAP2  LANG'DATE'SEP,S,1
      MAP2  LANG'TIME'SEP,S,1
      MAP2  LANG'DATA'SEP,S,2
      MAP2  LANG'PPN'LEFT,S,1
      MAP2  LANG'PPN'RIGHT,S,1
      MAP2  LANG'CHARSET,F,6
      MAP2  LANG'YES'WORD,S,6
```

```
MAP2  LANG'NO'WORD,S,6
MAP2  LANG'YES'CHAR,S,1
MAP2  LANG'NO'CHAR,S,1
MAP2  LANG'WORD'CHARS,S,30
MAP2  LANG'ULC,S,30
MAP2  LANG'COL,X,128
MAP2  LANG'USR,X,30
MAP2  LANG'SPARE,X,30
```

The information returned is defined as:

LANG'NAME1	A string variable containing the primary name of the current language.
LANG'NAME2	A string variable containing the alternate name of the current language.
LANG'EXTENSION	A string variable containing the three character extension to be used for message files related to the current language.
LANG'CURRENCY'SYM	A string variable containing the currency symbol characters used by the current language.
LANG'CURRENCY'POS	A floating point variable containing the encoded currency position. Will be set to 0 if the symbol comes before the value, or 1 if after.
LANG'CURRENCY'SPC	A floating point variable containing the currency spacing.
LANG'CURRENCY'SIZE	A floating point variable containing the number of digits making up the currency amount.
LANG'THOUSANDS	A string variable containing the character to be used to separate groups of digits to mark the thousands place.
LANG'DECIMAL	A string variable containing the character to be used as the decimal point.
LANG'DATE'FORMAT	A floating point variable specifying date format: 0 = month-day-year 1 = day-month-year 2 = year-month-day
LANG'TIME'FORMAT	A floating point variable specifying time format: 0 = 12 hour 1 = 24 hour
LANG'DATE'SEP	A string variable containing the character to be used to separate the fields of the date.
LANG'TIME'SEP	A string point variable containing the character to be used to separate the fields within a time.
LANG'DATA'SEP	A string point variable containing the character to be used to separate elements of data.
LANG'PPN'LEFT	A string variable containing the character to be used as the left marker for a PPN.
LANG'PPN'RIGHT	A string variable containing the character to be used as the right marker for a PPN.

LANG'CHARSET	A floating point variable containing the character set number to be used with the current language.
LANG'YES'WORD	A string variable containing the word for "yes."
LANG'NO'WORD	A string variable containing the word for "no."
LANG'YES'CHAR	A string variable containing the single character for "yes."
LANG'NO'CHAR	A string variable containing the single character for "no."
LANG'WORD'CHARS	A string variable containing all the characters, other than A-z, to be considered part of a word.
LANG'ULC	A string variable containing up to 15 pairs of upper and lower case characters that are equivalent.
LANG'COL	An unformatted variable containing the collating sequence to be used for the current language.
LANG'USR	An unformatted variable containing the user-definable portion of the current language.
LANG'SPARE	Reserved by Alpha Micro.

HOW TO USE GTLANG

Here is part of a sample program that demonstrates one use of the GTLANG call:

```
MAP1  GTLANG'MAP
      MAP2  LANG'NAME1,S,20
                .           ! Other MAP statements (see above)
      MAP2  LANG'SPARE,X,30
XCALL GTLANG,STATUS,GTLANG'MAP
PRINT "Welcome to the Accounting General"
PRINT "Ledger Update program!"
PRINT "Enter the current date, using"
IF LANG'DATE'FORMAT = 0 THEN &
   PRINT " month-day-year";
IF LANG'DATE'FORMAT = 1 THEN &
   PRINT " day-month-year";
IF LANG'DATE'FORMAT = 2 THEN &
   PRINT " year-month-day";
INPUT " format: "
```

In this program, we check the current language to see what format that language uses to express the date, so the person entering the date on the terminal is instructed how to properly enter it. Using the GTLANG subroutine, you can tailor your programs to work with more than one language. For example, the beginning of the run of the above program might look like this:

```
RUN GTDATE [RETURN]
Welcome to the Accounting General Ledger Update program!
Enter the current date, using day-month-year format:150285 [RETURN]
```

Chapter 9 - TRMCHR
Gets Terminal Characteristics

This chapter discusses:

- What is TRMCHR?
- The TRMCHR format
- How to use TRMCHR

WHAT IS TRMCHR?

TRMCHR allows you to find out many of the characteristics of the terminal on which your program is currently running. It returns information such as the number of rows and columns available on the screen; the number of colors, including the foreground and background colors; and flags describing the other features which your terminal supports.

It is not necessary to load TRMCHR into system or user memory, but, since it is fully re-entrant and re-usable, you may do so for increased speed.

TRMCHR FORMAT

```
XCALL TRMCHR,status,TRMCHR'MAP
```

Status is a floating point variable in which the success (0) or failure (-1) of the operation is returned.

TRMCHR'MAP is a MAP1 level area which will receive the descriptive information about the terminal. It must be defined like this:

```
MAP1  TRMCHR'MAP
      MAP2  TRMCHR'FLAGS,F,6
      MAP2  TRMCHR'ROWS,F,6
      MAP2  TRMCHR'COLS,F,6
      MAP2  TRMCHR'COLORS,F,6
      MAP2  TRMCHR'FORE,F,6
      MAP2  TRMCHR'BACK,F,6
      MAP2  TRMCHR'WINROW,F,6
      MAP2  TRMCHR'WINCOL,F,6
```

The information returned is defined as follows:

TRMCHR'FLAGS

TRMCHR'FLAGS returns one or more of the following:

1	Alternate page support
2	Block fill support
4	Column insert/delete support
8	Multiple color support
16	Function key translation support
32	Is an Alpha Micro terminal
64	Has a status line
128	Horizontal split screen support
256	Erase to end of line support
512	Erase to end of screen support
1024	Underscore attributes support
2048	Blinking support
4096	Dim video support
8192	Reverse video support
16384	Character insert/delete support
32768	Line insert/delete support
65536	Local printer support
131072	Settable 80/132 column support
262144	Selectable smooth scroll support
524288	Has box drawing commands
1048576	Has "mode" type attributes
2097152	Has AM-70 style color commands
4194304	Has "no space" attribute commands

The number returned is the total value of all of the above conditions that are true for your terminal. For example, if the number returned is 4194, you can deduce you have an Alpha Micro terminal with a status line that supports block fill and dim video (2 + 32 + 64 + 4096 = 4194).

TRMCHR'ROWS

A floating point variable that contains the number of rows (horizontal lines) the terminal supports.

TRMCHR'COLS

A floating point variable that contains the number of rows (vertical lines) the terminal supports. May vary, as some terminals support settable columns.

TRMCHR'COLORS

A floating point variable that contains the number of colors the terminal supports. Monochrome terminals will report 0.

TRMCHR'FORE	A floating point variable that contains the number of the current foreground color. Monochrome terminals will report 0. The color values are: 0 Black 1 White 2 Blue 3 Magenta 4 Red 5 Yellow 6 Green 7 Cyan
TRMCHR'BACK	A floating point variable that contains the number of the current background color. It uses the same values as TRMCHR'FORE, above.
TRMCHR'WINROW	A floating point variable that contains the number of rows in the current window. Reserved for future use.
TRMCHR'WINCOL	A floating point variable that contains the number of columns in the current window. Reserved for future use.

HOW TO USE TRMCHR

Here is a small program that uses TRMCHR:

```
MAP1  TRMCHR'MAP
      MAP2  TRMCHR'FLAGS,F,6
      MAP2  TRMCHR'ROWS,F,6
      MAP2  TRMCHR'COLS,F,6
      MAP2  TRMCHR'COLORS,F,6
      MAP2  TRMCHR'FORE,F,6
      MAP2  TRMCHR'BACK,F,6
      MAP2  TRMCHR'WINROW,F,6
      MAP2  TRMCHR'WINCOL,F,6

XCALL TRMCHR,STATUS,TRMCHR'MAP

IF TRMCHR'FLAGS AND 8 THEN GOSUB COLOR'PRINT  ! Does terminal
PRINT "This was a test."                      ! have color?
END

COLOR'PRINT:

     PRINT TAB(-3,4)     ! Set red background color
     PRINT TAB(-2,1);    ! Set white foreground color
     RETURN
```

In the above program, we see (by using the logical AND operator) if 8 (the flag indicating color support) is one of the TRMCHR'FLAGS. AND checks to see if 8 was one of the numbers that was added to get the value of TRMCHR'FLAGS. If so, we go to the COLOR'PRINT subroutine, and set up colors for the output.

Using TRMCHR, you can write programs that "adapt" to whatever terminal they are run on, taking advantage of the capabilities of more sophisticated terminals without "crashing" on less sophisticated terminals.

Chapter 10 - Terminal Input and Display Subroutines

This chapter discusses the following calls:

- ACCEPT—gets arbitrary data from the terminal
- ECHO—turns on terminal echo
- NOECHO—turns off terminal echo

ACCEPT

Provides a way to accept arbitrary data from the terminal, including control characters and others that are normally trapped by the system. It also provides a way of accepting a single character rather than requiring a RETURN before any input is accepted. The format is:

```
XCALL ACCEPT,CHAR
```

CHAR is a floating point variable that will receive the ASCII value of the character entered.

ACCEPT takes any character, but does no editing on it. It is fully re-entrant, and may be loaded into system memory.

Your program MUST call the NOECHO subroutine (see below) before using ACCEPT. NOECHO places the terminal in Image Mode, which protects the terminal against input errors (since ACCEPT allows anything to be input, including things the terminal might not be able to display).

ECHO

Used to turn on the terminal's echoing (the normal character display mode of your terminal), and to take the terminal out of image mode (the display mode used for special input). The format is:

```
XCALL ECHO
```

The END operation automatically resets the echo and takes the terminal out of image mode. ECHO is fully re-entrant, and may be loaded into system memory.

NOECHO

Used to turn off the terminal's echoing and put the terminal into image mode. NOCECHO is necessary for ACCEPT and other subroutines which use image mode input to prevent the terminal screen from being destroyed by errors in operator input, and to allow single characters to be accepted. The format is:

```
XCALL NOECHO
```

NOECHO is fully re-entrant, and may be loaded into system memory.

Chapter 11 - Miscellaneous XCALLs

This chapter discusses the following XCALLs:

- IDTIM - Converts time from string to internal format
- ODTIM - Converts time from internal format to string
- RENAME - Renames an AMOS file
- SLEEP - Puts terminal to sleep for a specified period
- STRIP - Removes trailing blanks from a string
- MSGLOG - Sends a message to the system event log.
- LKINFO - Returns job and terminal information of a user that has a record locked under LOKSER.

IDTIM

Converts a date and/or a time from symbolic (string) format to internal format (as returned by the DATE and TIME functions of AlphaBASIC). It is not necessary to load IDTIM into system or user memory, but, since it is fully re-entrant and re-usable, you may do so for increased speed. The format is:

```
XCALL IDTIM,dstrng,idate,itime,flags,status
```

dstrng A string variable to contain the date and/or time you wish to encode.

idate A floating point variable to contain the encoded date when done. Date is returned in separated format.

itime A floating point variable to contain the encoded time when done. Time is returned as the number of seconds since midnight.

flags A floating point variable which can contain these binary encoded flags:
1 - Do not scan for date
2 - Do not scan for time

status A floating point variable which returns a 0 for success or a -1 for failure of the operation.

All leading spaces are ignored when parsing the string. If both the date and the time are specified, the time must follow the date in the string. The date must agree with the format of the currently selected language. Spaces or dashes (-) may be substituted for the / separator in the date.

Two-digit years are assumed to be in the 20th century (19xx). Use four-digit years for years past 1999.

IDTIM works properly for dates between March 1, 1900 and December 31, 2099, inclusive.

If the time field is selected, it must contain a string with the format “hh:mm:ss” (the seconds are optional). If AM or PM is added, it must come after the time specification.

ODTIM

Converts a date and/or a time from internal format (as returned by the DATE and TIME functions of AlphaBASIC) to symbolic (string) format. It is not necessary to load ODTIM into system or user memory, but, since it is fully re-entrant and re-usable, you may for increased speed.

ODTIM works properly for dates between March 1, 1900 and December 31, 2099, inclusive. Its format is:

```
XCALL ODTIM,dstrng,odate,otime,flags
```

dstrng is a string variable which will contain the decoded date and time. This variable must be at least 45 bytes long.

odate is a floating point variable which contains the date to decode, as returned by the DATE function. If 0, the current date is used.

otime is a floating point variable which contains the time to decode, as returned by the TIME function.

flags is a floating point variable. To determine its value, first decide which options you want from the lists in the next section.

Ordering and Formatting Options

0	Omit date from the output; ignore all other date formatting flags.
9	Omit time from the output; ignore all other time formatting flags.
14	Affects numerical output for day, month, and year only. If on, leading zeros are suppressed on all elements. This produces output suitable for use in an isolated message. If off, leading zeros are added if appropriate, producing output of a constant width that is useful for printing tables. Note that full month and weekday texts are never columniated, and time formats always have leading zeros. Overridden by Option 18. See also Option 19.
26	Output the time before the date, else the time will follow the date.

DATE FORMATTING

Date Punctuation

7-8 Date punctuation control (can be overridden by Option 21):

Both off	Use dashes, as 20-Feb-82
7 only	Use spaces, as in 20 Feb 82; if Option 6 is on and Option 3 is off, add a comma, as in February 20, 1990
8 only	Use slashes, as 2/20/82.
Both on	Use the character defined as the date separator in the job's language definition file.

20 Do not output any commas in the date string. Overrides Option 2 and Option 6.

21 Do not output date punctuation. Overrides Options 7 and 8.

Date Ordering

6 Output the day of the month after the month, otherwise output it before. Ignored if Options 7 or 8 are set. A comma is appended if the day is immediately followed by the year and the month is not output as a number.

16 If set, the year precedes the day and month (which are printed in the order specified by Option 6).

Date Items Output

1 Output the day of the week.

23 Do not output the day. Overrides Options 6 and 19.

24 Do not output the month. Overrides Options 3 and 4.

25 Do not output the year. Overrides Options 5 and 16.

Weekday Options

2 Use the full text for the weekday, otherwise use a three-letter abbreviation. A comma will be added if alphabetic output is generated and the date follows the weekday. Requires Option 1. Option 22, if set, overrides this option.

22 Output the weekday as a number (0-6) as the first element of date output. Overrides Option 2, requires Option 1 to be set.

Day Options

19 Add English day suffix (st, nd, rd, th) to the numerical day output. Forces Option 14 on for day output.

Month Options

3 Output the month as a number 1-12; ignore Option 4.

4 Use the full text for the month, otherwise use a three-letter abbreviation.

Year Options

5 Output a four-digit year, otherwise use a two-digit year. Option 14 may disable leading zeros for two-digit years; Option 18 may force them on.

18 Force leading zeros in two-digit years for years in the range xx00-xx09. Overrides Option 14.

TIME FORMATTING

11 Use 12-hour and AM/PM, otherwise use 24-hour time.

15 The time value is supplied in separated format. Otherwise the time value is in internal format.

28 If 24-hour clock, output "hrs" after the time string. Otherwise ignored.

Hours Options

12 Do not output a separator between the hour and the minute values.

27 Affects hours output: if 12-hour clock, suppress any leading zero. If 24-hour clock, ignored.

Seconds Options

10 Omit seconds; otherwise include seconds preceded by the time separator.

Punctuation Options

13 Force a colon as the time separator, otherwise use the character defined as the time separator in the job's language definition file.

17 Do not output separator between the minute and the second.

Determining Flag Values

To determine the value of ***flags***, follow these steps:

1. Decide which options you want from the lists above, and note the number of each option.
2. For each number in step 1, add the corresponding values in the table below.
3. The grand-total value is the value of the flags variable.

Option	Value	Option	Value
0	1	15	32768
1	2	16	65536
2	4	17	131072
3	8	18	262144
4	16	19	524288
5	32	20	1048576
6	64	21	2097152
7	128	22	4194304
8	256	23	8388608
9	512	24	16777216
10	1024	25	33554432
11	2048	26	67108864
12	4096	27	134217728
13	8192	28	268435456
14	16384		

A zero in ***flags*** produces columniated output like this:

```
24-Mar-84 01:06:03
```

For convenience, a -1 in the flags variable is taken as the value 64366, which produces output in the form:

```
Tuesday, March 24, 1984 01:06:03 AM
```

Depending on the currently selected language and the contents of the language definition file associated with that language, this format may output the date in day-month format, or the time in 24-hour format. Otherwise, the language settings have no effect on this subroutine.

RENAME

Provides a way to rename disk files from within AlphaBASIC. The format is:

```
XCALL RENAME,oldnam,newnam,status
```

Oldnam is a valid AMOS filespec string specifying the current name of the disk file.

Newnam is a valid AMOS filespec string specifying the new name.

Status is a floating-point variable to be filled with 0 if the rename is successful, or non-zero if it fails. RENAME fails if the oldnam does not exist, or if the newnam already exists (it will not overwrite an existing file).

RENAME is fully re-entrant, and may be loaded into system memory.

SLEEP

Causes your program to wait for a specified number of seconds. This period can range from .0001 seconds to 4.97 days. It is not necessary to load SLEEP into system or user memory, but, since it is fully re-entrant and re-usable, you may for increased speed. The format is:

```
XCALL SLEEP,amount
```

Amount is a floating point variable which contains the number of seconds you wish your program to sleep. You can use fractional values. For example:

```
XCALL SLEEP,10
```

causes your program to sleep for ten seconds.

STRIP

Whenever random-length strings are being handled, it often becomes necessary to be able to remove the trailing blanks from such a string. The STRIP subroutine provides a convenient method for doing just that, at a speed about 100 times faster than can be done in BASIC. The format is:

```
XCALL STRIP,STRING
```

String is the string from which the trailing blanks are to be deleted. For example:

```
READ #1,TEXT'LINE$
PRINT "The length of the string is: ",LEN(TEXT'LINE$)
XCALL STRIP,TEXT'LINE$
PRINT "The length after STRIP: ",LEN(TEXT'LINE$)
```

If the line in the file had spaces at the end of the data, a run of the above program might look like this:

```
The length of the string is: 76
The length after STRIP: 65
```

If a string of blanks is specified, a null (empty) string is returned. STRIP makes no use of other subroutines. It is fully re-entrant and may be loaded into system memory.

MSGLOG

Lets you send a message with a code of 1 to 999 to the system event log (LOGGER). It is not necessary to load MSGLOG into system or user memory, but you may do so for increased speed. The format is:

```
XCALL MSGLOG,text,code,status
```

text	A string of up to 45 characters to be placed in the log file.
code	A floating point, binary, or integer value (1-999) that represents the error code you want to send to the event log.

status A four-byte binary variable to contain the status code on return. The values that can be returned are:

- 0 No error.
- 1 Invalid type for Argument 1 (***text***).
- 2 Invalid type for Argument 2 (***code***).
- 3 The error code was greater than 999 or less than 1.

If all three arguments are not present, or if the STATUS variable is not mapped as binary, the XCALL is ignored with no error message.

LKINFO

The LKINFO subroutine returns the job and terminal information of a user who has a record locked under LOKSER. You can use this when an attempted access fails because the record is locked. LKINFO's format is:

```
XCALL LKINFO, file, status, jobname, termname, jobuser
```

file Either the channel number of the channel on which the file is open, or a null-terminated file name which the subroutine will attempt to OPEN. When using the channel number, it should be a variable type supported by $GTARG (one- to four-byte binary or integer, AMOS floating point, or IEEE four-byte or eight-byte floating point).

You will need to use a file name if the program's previous attempt to OPEN the file failed. A failed OPEN in AlphaBASIC does not allocate a valid channel number. So, you must pass the file name to LKINFO. LKINFO opens the file, processes it, then closes the file again.

status An AMOS floating point variable to hold the returned value. Possible returned values are:

- 0 Success. Further variables hold user information.
- 1 The DDB error code was not D$ERIU or D$EFIU.
- 2 File was not locked.
- 3 Cannot determine JCB.
- 4 File does not exist (only when using file name).
- 5 JCB is invalid.
- 6 File is not opened on given channel.

If status is not equal to 0, all other return fields are set to null.

jobname A six-byte or longer string to hold the job name of the JCB owning the lock.

termname A six-byte or longer string to hold the name of the terminal attached to the job which owns the lock.

jobuser A 20-byte or longer string to hold the user name of the job owning the lock.

If you execute this call after a failed attempt to open a file, the file channel number is not available. Therefore you may need one ON ERROR GOTO routine from an OPEN statement leading to an XCALL LKINFO using the file name, and another ON ERROR GOTO routine after the file is OPENed successfully leading to an XCALL LKINFO using the channel number.

Context switches could occur between the time AMOS detects that the file or record is locked and the point that LKINFO is run, and the file/record may no longer be locked. If so, AlphaBASIC would report “xxx in use”, but LKINFO would return a status code of 2 (E.NOT.LOCKED). Both are correct, and your program must handle this situation correctly, possibly by retrying the statement that gave the error in the first place.

Chapter 12 - FNDUSR Locates a User Description

This chapter discusses:

- What FNDUSR does
- How you use FNDUSR
- The information returned by FNDUSR

This XCALL is only available on AMOS/32 1.0 and AMOS/L 2.0 and later versions. If you have an earlier version of the operating system, this call will not work.

WHAT IS FNDUSR?

FNDUSR locates descriptive information about a system user. You call the subroutine with the name of the user you wish to receive information on, and it returns a block of descriptive information.

It is not necessary to load the FNDUSR subroutine into system or user memory. You may do so if you want increased access speed.

FNDUSR FORMAT

```
XCALL FNDUSR,user'name,status,user'map
```

User'name is a string specifying the user name you want descriptive information on. If this string has a length of 0, FNDUSR assumes the user name of the caller's job.

Status is a floating point variable that tells you the success or failure of the user lookup operation. The possible return codes are:

0	User was located successfully.
1	Unable to locate user.
-1	File error reading DSK0:USER.SYS[1,2].
-2	Insufficient memory to run FNDUSR.SBR.

User'map is a variable to receive the information on the user. It must be defined as:

```
MAP1  USER'MAP
      MAP2  USER'NAME,S,20
      MAP2  USER'FLAGS,F,6
      MAP2  USER'ROOT'PPN,S,10
      MAP2  USER'ROOT'DEVICE,S,10
      MAP2  USER'MAIL'PPN,S,10
      MAP2  USER'MAIL'DEVICE,S,10
      MAP2  USER'MAIL'CPU,S,10
```

```
MAP2  USER'PRIV,F,6
MAP2  USER'CLASS,F,6
MAP2  USER'EXP,F,6
MAP2  USER'CPU'TIME,F,6
MAP2  USER'CONNECT'TIME,F,6
MAP2  USER'KRAM,F,6
MAP2  USER'DISK'READS,F,6
MAP2  USER'DISK'WRITES,F,6
MAP2  USER'PAGES'PRINTED,F,6
MAP2  USER'DEFAULT'PROTECTION,F,6
MAP2  USER'DEFAULT'PRIORITY,F,6
MAP2  USER'DEFAULT'LANGUAGE,S,20
MAP2  USER'DEFAULT'PROMPT,S,20
```

The User'map Fields

The fields defined for the user'map variable are:

USER'NAME	A 20-byte string that contains the user name of the requested user.
USER'FLAGS	A floating point variable that contains flags describing various attributes of the user. Not currently used.
USER'ROOT'PPN	A 10-byte string that contains the number of the user's root account. It is formatted with full punctuation (for example, "[377,12]").
USER'ROOT'DEVICE	A 10-byte string that contains the device and unit number of the user's root account.
USER'MAIL'PPN	A 10-byte string that contains the account number of the user's mail account. It is formatted with full punctuation (for example, "[123,0]").
USER'MAIL'DEVICE	A 10-byte string that contains the device and unit number of the user's mail account.
USER'MAIL'CPU	10-byte string that contains the CPU number of the user's mail account.
USER'PRIV	A floating point variable which contains the default privileges for the user. The privileges are formatted as a series of "bit" values that are summed to create the value returned in USER'PRIV. The values assigned to each of the bits means the user may: 1 Read system memory. 2 Write system memory. 4 Read physical disk blocks. 8 Write physical disk blocks. 16 Run diagnostic programs. 32768 Change privileges.

USER'CLASS

A floating point number from 0 to 100 describing the class of the user. While the meaning of CLASS is up to you, here are some guidelines:

0-25	Restrict the user to operating within a turnkey application, such as AlphaMENU. Give read access to system library accounts and read/write access to own account. No other access allowed.
26-50	Give access to a wider variety of programs, but still restrict access to other accounts. Give read access only to system library accounts and other accounts in the same project series, but write access only to own account.
51-75	Give read access to all accounts, but restrict to writing in same project series.
76-100	Same read/write access as above, but allow greater access to potentially damaging programs.

USER'EXP

A floating point number describing, on a scale of 0 to 100, the expertise level of the requested user. The expertise level is used to modify the level of detail or explanation which a user receives. For example, a user with a low level of expertise might receive a more detailed error message than a user of more expertise.

USER'CPU'TIME

A floating point number containing the total number of CPU seconds this user had used, as of the last time the user logged off the system.

USER'CONNECT'TIME

A floating point number containing the total number of seconds the user had been logged in, as of the last time he logged off the system.

USER'KRAM

A floating point number containing the total number of kiloRAM-seconds the user had used, as of the last time the user logged off the system. KiloRAM-seconds is the product of the amount of memory used by the user (expressed in kilobytes) and the number of seconds the user was logged in.

USER'DISK'READS

A floating point number containing the total number of disk reads the user had performed, as of the last time the user logged off the system.

USER'DISK'WRITES

A floating point number containing the total number of disk writes the user had performed, as of the last time he logged off the system.

USER’PAGES’PRINTED	A floating point number containing the total number of pages printed by the user, as of the last time the user logged off the system.
USER’DEFAULT’PROTECTION	Reserved for future use.
USER’DEFAULT’PRIORITY	A floating point number containing the job priority assigned at log in.
USER’DEFAULT’LANGUAGE	A 20-byte string containing the language the user was assigned at log in.
USER’DEFAULT’PROMPT	A 20-byte string containing the prompt assigned to the user at log in.

Chapter 13 - TCP
TCP Networking in AlphaBASIC

AlphaTCP is Alpha Microsystems' implementation of the standard DARPA TCP/IP family of protocols. With AlphaTCP, your applications can communicate with other applications running on a large variety of systems. The TCP XCALL gives your AlphaBASIC applications access to the TCP protocol.

The TCP XCALL requires AlphaTCP 1.4 or later and AMOS 2.3 or later. The AlphaTCP TAME server must be active on the system. See the *AlphaTCP Administrator's Guide* for information on installing AlphaTCP and setting up servers.

TCP PROGRAMMING OVERVIEW

The next few sections provide a very brief overview of TCP application programming. You should be familiar with these concepts before you start using the TCP XCALL.

Client/Server Paradigm

Most TCP/IP software is written according to the client/server paradigm. The concept is that *server* programs provide valuable services and functions to *client* programs which contact them as needed.

Servers usually start at boot time and never exit. They passively listen in the background until something is requested of them; they do not normally have a physical terminal attached. Because they must be contacted, they listen on *well-known port numbers*. If you imagine the U.S. telephone system, servers will always be contacted by a well-known number (such as *0* or *911)*, which can also be associated with a well-known service name (*operator* or *emergency*). Port numbers and service names are explained below. In the UNIX world, servers are often referred to as *daemons*.

Clients usually contact a server to perform their operation(s). They often have a physical terminal attached, but not always. Since they do not usually need to be contacted themselves, they may initiate their conversation on whatever port the system assigns them at the time. This is known as an *ephemeral port*. Again, if you imagine the U.S. telephone system, it matters little which telephone number you are calling *from*. (There might be an argument for the emergency number 911, but just like 911 a server can identify where you are originating from).

IP Addresses and Port Numbers

A TCP/IP communication endpoint is defined by a *protocol*, an *IP address*, and a *port number*. A virtual communication channel exists between two applications when each application's endpoint references the other application's endpoint using a common protocol. This association is known as a *5-tuple*, because it involves two IP addresses, two port numbers, and one protocol.

- The *protocol* is the method used to communicate between applications on systems. This is not the physical interface, such as Ethernet; instead, it is the language spoken across the interface, much like speaking English or French during a telephone conversation. In the same way that both sides of this conversation speak the same language, both sides of a TCP/IP connection must speak the same protocol. The most common protocols are *TCP* and *UDP*. While each protocol has its own benefits, the TCP XCALL supports TCP only. TCP is a reliable protocol which not only handles the intricacies of data delivery over potentially unreliable paths, but adjusts itself to the varying bandwidth available when networks are connected using slower links.
- TCP/IP uses the *IP address* to identify a system. The IP address is usually shown as four decimal values, each between 0 and 255, separated by periods. . An example of an IP address is *127.0.0.1*, which is the loopback address (an address that points back to your local system so network applications can communicate without a network). Continuing the telephone example, the IP address is comparable to the telephone number.
- The *port number* identifies a place on the system from which data may be read or written. Often this means a running application; however, an application may have many communication channels open. Port numbers range from 1 to 65535. Port numbers under 1024 often have a special meaning. Certain values are assigned to various standard applications. These are known as *well-known* port numbers. Most systems allocate their temporary port numbers starting at 1024. Historically, programs using port numbers under 1024 on a UNIX system could only be started by the system administrator. This made them more secure. However, with the proliferation of PCs this trait can no longer be trusted.

Note that an actual conversation is defined by the complete 5-tuple. Thus, it is perfectly valid to see many separate conversations active using the same port numbers, as long as some portion of the association is unique.

Service Names

The well-known ports are assigned *service names* to make them easier to remember. Whenever a TCP/IP application encounters a service name, it looks it up to get the real port number associated with it. Most often your programs will work with service names rather than port numbers, choosing to connect to or provide a *service*, while underneath it all the system handles the actual *port number*. For example, the telnet service uses the well-known port number 23. If you are still curious, examine the file TCP:SERVIC. for the service names AlphaTCP knows about.

Host and Domain Names

Since a series of digits is difficult to remember, systems are assigned names in addition to IP addresses. This is usually done in a hierarchy of *host name* and *domain name*. The full specification of host name with domain name is known as the system's *fully qualified domain name* (FQDN). Whenever a TCP/IP application encounters a name, it looks it up to find its IP address. If the name is not fully qualified, portions of the domain name are appended to find a match.

Much like an IP address, the FQDN is divided into sections separated by periods. Unlike IP addresses, there is no fixed number of sections, allowing a flexible naming convention which can fit the organizational structure of a site. The domain name may contain many sections. From left to right,

sections of the FQDN represent higher levels of administration, with as many levels as an organization sees fit.

For example, the name *orders.widgit.com* can represent the order entry system at Widgit, Inc.'s headquarters. Taking this further, *pc.tx.widgit.com* can represent the Production Control system at their plant in Texas, while *mail.tx.widgit.com* represents the mail system at that same plant.

The *com* section is a virtual *root domain* that represents commercial entities. A university would use *edu* instead. International sites use a different style. More information about root domains can be found in the *AlphaTCP Administrator's Guide*, or most other TCP/IP publications.

TCP XCALL OVERVIEW

The following sections provide an overview of the programming implementation used by the TCP XCALL.

Procedural Versus Event-Driven

There are two primary models of software development in use these days: procedural and event-driven.

- The procedural model follows a step-by-step flow of control. Most procedural TCP/IP programming is written with *blocking synchronous* calls. In other words, the call does not return until satisfied. For example, a procedural server writes a welcome message and waits for the client to issue its request. If other writes were done previously, the welcome message write may pause. Nothing else can be done until the input is provided, at which point the program performs its next step. Error conditions must be addressed in every section of the program. Modifying the behavior of a procedural program can be difficult. Procedural programs tend to interact with their environment in many different places in the code, the execution of which depends on past operations.
- The event-driven model follows no strict flow. Instead, the program reacts to stimuli known as *events*. Because events may come in at different times, in different order, most event-driven TCP/IP programming is written with *non-blocking asynchronous* calls. In other words, the calls always return right away. Some calls simply initiate an operation, while others return an error if they are called before they should be. You are notified of call completion and readiness through processing of events. In the previous example, you will know when it's OK to write through an event. You will know when input is available in the same manner. Your application is free to perform other tasks in between. Event-driven programs tend to have a main loop of execution where interaction with their environment takes place, and state flags to control behavior from past operations.

The TCP XCALL uses the event-driven model, which is a newer style popularized by the advent of GUI operating systems.

Features

The TCP XCALL provides a large number of calls useful in network programming, plus several support functions. Functions are provided to:

- Establish, accept, and close network connections
- Read and write data
- Transfer control of a connection to another job
- Return useful connection information
- Dynamically spawn jobs
- Pass information between jobs
- Convert path and file names to AMOS format in a standard way

Program Operation

As a client, your application first attempts to establish a connection. As a server, your application begins listening for connections, then accepts them as they are requested.

Once a connection is established, operation of a client or a server is pretty much the same. The application checks for events, retrieves the active events and services them as needed, then notifies TCP that event processing is complete. This is repeated until the connection is no longer needed. At that point, the connection can be closed. A client application would likely exit. A server would likely wait for the next connection request. Features are provided to create applications that handle multiple connections simultaneously, either by timesharing themselves or by creating new jobs dedicated to servicing a connection.

Variable Types

The TCP XCALL is reasonably flexible with the variable types passed in calls, provided the variable type used can handle the maximum values expected. Many value arguments may be passed as strings, floating point, or binary values. There is often a performance benefit in using the binary formats. In all cases, a 4-byte binary variable will work for numeric storage. In some cases listed below you may even use a 2-byte binary variable.

TCP XCALL PROGRAMMING

This section covers use of the various function calls provided by the TCP XCALL. You can find sample programs which use the TCP XCALL in DSK0:[7,6] (BAS:).

The descriptions below refer to a *session*. In many instances this refers to a TCP *connection* handled through the XCALL, but a connection does not need to exist. Examples of this are connections pending, being listened for, or recently closed. Though a network connection does not exist in these instances, a *session* with the XCALL still exists while the *handle* (see below) is still valid. This is the distinction between *session* and *connection*.

Include File

The include file TCP.BSI contains definitions for function and error codes, events, and flags used in various calls. You should ++INCLUDE this file near the top of your program. All of the following examples use the definitions from the include file.

Detecting Events

Normally, event-driven programming would be handled by specifying to the operating system which functions to call to handle which events. AlphaBASIC is not designed to support this. Instead, certain calls to the TCP XCALL, coupled with proper programming style, combine to allow an AlphaBASIC program to be event-driven.

The key to event-driven AlphaBASIC is to ***avoid non-event-driven wait states***. There will be times when your program has no further work to perform and must wait for some external activity to take place. In many programs the usual solution is to wait for keyboard input. Because keyboard input is only one of the possible activities that should awaken you, ***this should not be done in an event-driven application***. Instead, use the following call:

```
XCALL TCP,TCP'WAIT'FOR'ACTIVITY,ErrCode,Activity
```

This causes the job to wait until there is something to do. ***Activity*** should be a 2-byte binary variable. Upon return, it contains which types of activity are present. Each currently present activity is represented by a bit in this variable. You may AND the following values with Activity in an IF statement to test for TRUE or FALSE:

TCP'ACTIVITY'KEYBOARD	Keyboard input is present
TCP'ACTIVITY'MESSAGE	ITC message(s) present
TCP'ACTIVITY'EVENTS	TCP events need servicing
TCP'ACTIVITY'USER1	User-defined signal #1 seen
TCP'ACTIVITY'USER2	User-defined signal #2 seen
TCP'ACTIVITY'ALARM	Alarm clock timer expired
TCP'ACTIVITY'SIEXIT	Job is being terminated

For example, you could use the following code fragment to check the type of event and go to the proper section of your application:

```
Main'Loop:
   XCALL TCP,TCP'WAIT'FOR'ACTIVITY,ErrCode,ActivityFlags
   IF (ActivityFlags AND TCP'ACTIVITY'SIEXIT) GOTO Exit'Program
   IF (ActivityFlags AND TCP'ACTIVITY'KEYBOARD) CALL Service'Keyboard
   IF (ActivityFlags AND TCP'ACTIVITY'MESSAGE) CALL Service'AlphaNET
   IF (ActivityFlags AND TCP'ACTIVITY'EVENTS) CALL Service'TCP
   IF (ActivityFlags AND TCP'ACTIVITY'USER1) CALL Service'User1
   IF (ActivityFlags AND TCP'ACTIVITY'USER2) CALL Service'User2
   IF (ActivityFlags AND TCP'ACTIVITY'ALARM) CALL Service'Timer
   IF (ActivityFlags AND TCP'ACTIVITY'SIEXIT) GOTO Exit'Program
   GOTO Main'Loop
```

You can have your program check for TCP events while it is doing other processing by using the following call:

```
XCALL TCP,TCP'QUERY'EVENTS,ErrCode,HaveEvents,AutoSet
```

Upon return, ***HaveEvents*** is set to zero if no events are present, or non-zero otherwise. The ***AutoSet*** parameter is optional and may be any variable type. If an argument is present for AutoSet, then HaveEvents will be set whenever TCP events need servicing, without requiring a TCP'QUERY'EVENTS call again.

TCP'WAIT'FOR'ACTIVITY checks for any activity, including a TCP event (such as a new connection), but also including other activity, such as keyboard input. TCP'QUERY'EVENTS, however, checks only for TCP events, not other activity.

Setting User-Defined Signals

The TCP'WAIT'FOR'ACTIVITY call supports two user-defined signals. These may be set by a program running on another job. As no counter is maintained, be aware that setting these signals multiple times only results in a single notification for each. Once the application has received notification of the signal, the signal will be reset and ready once again.

You can use the SETUSI XCALL to set these signals. The format of the call is:

```
XCALL SETUSI,Job,Signal,Errval
```

Job is a string variable or constant containing the name of the job to receive the signal.

Signal is a value representing which user defined signal to set: 1 or 2.

Upon return, ***Errval*** contains: 0 for success, 1 for invalid jobname, or 2 for invalid signal.

Handles

Many calls involving a session (connection) either return or require a *handle*. This is simply a value TCP.SBR uses to identify the session. You never need to modify a handle yourself. Simply store the handle when it is provided to you, and pass it in calls requiring it. A handle is a 4-byte binary value. For efficiency and robustness, you should MAP your handle storage as such.

Process IDs

All jobs performing TCP related functions have a *process ID* associated with them. This is a convenient way for the system to control the jobs. It also provides a convenient way for your application to control other jobs. For instance, a spawned job that spawns its own jobs can reference its own process ID, its *parent* process ID, and all of its *child* process IDs. These may become important to you as you develop more complex networked applications. Process IDs are 2-byte binary values. It is more efficient to MAP storage for them to this size.

Process IDs exist in a maximum of 5 digits (65535), they are unique to a job, and are not reused until a system reboot or a wrap occurs. One handy use for them is in temporary filename construction.

Events

There are two types of TCP events to be concerned with: *global events* and *session events*. Global events are not specific to any session, while session events are. There are two ways to retrieve the active events; these are described in detail further on. Each event is an individual bit in a variable, allowing multiple events to be passed in a single call. Events are best stored in a 4-byte binary variable for speed.

Global Events

Global events primarily involve job control. There is one set of global events possible in a program. The global events you may see are:

TCP'GEVENT'CHILD	A spawned child job has exited
TCP'GEVENT'PARENT	Parent of this spawned job has exited
TCP'GEVENT'ATTENTION	Another job has requested attention
TCP'GEVENT'DOWN	TCP access subsystem being terminated

The first two events are described below in "Spawning Jobs." The third may be found below in "Attention Events." You will likely never see the fourth event, instead seeing the TCP'ERR'ENAVL error code returned in the ErrCode variable.

Session Events

Session events directly involve sessions. There is one set of session events for every session (whether active, pending, or listening). You may see these session events:

TCP'SEVENT'CONNECTED	Connection established or requested
TCP'SEVENT'DISCONNECTED	Connection properly closed
TCP'SEVENT'DROPPED	Connection was terminated
TCP'SEVENT'DATAIN	Data waiting to be read
TCP'SEVENT'DATAOUT	Data OK to write
TCP'SEVENT'RECORD	A record or line is ready
TCP'SEVENT'HANDOFF	Control of session has been requested

After you receive a disconnect or drop event, you should still close down the session to free its resources. See "Closing Sessions," below.

Retrieving Events

Events may be retrieved on a session-by-session basis. In addition, if you are controlling many sessions, you may choose to retrieve events for all sessions. To retrieve the global events, plus the events for one session, use the following call:

```
XCALL TCP,TCP'GET'EVENTS,ErrCode,Handle,Gevents,Sevents
```

Handle identifies the session, ***Gevents*** receives the current global events, and ***Sevents*** receives the session's active events.

To retrieve just the global events, pass a Handle value of zero, and you may leave off the Sevents parameter.

To retrieve events for all sessions you control, use the following call:

```
XCALL TCP,TCP'GET'ALL'EVENTS,ErrCode,Gevents,EventArray,FillCount
```

As before, ***Gevents*** receives the global events. However, the array ***EventArray*** receives the handle and events for all sessions which have events to process. ***FillCount*** receives the number of array entries filled in by the call. The array should contain enough entries to support events on all sessions at once, though it will handle smaller arrays properly. The format of the array is as follows:

```
MAP1 EventArray(nnn)
  MAP2 Handle,B,4
  MAP2 Events,B,2
```

Event Completion

Once you have completed servicing the currently set global events, or the events for a session, you must notify TCP.SBR that more events may be processed. To do that, use the following call:

```
XCALL TCP,TCP'EVENTS'DONE,ErrCode,Handle
```

In the case of global event completion, simply pass a ***Handle*** value of zero.

It is important to perform this call *only* when events were present, not when retrieved events were zero. Do not perform this call for each individual event, but only after processing all events in the variable. In the case of multiple session polling, this call should be performed once for each session in the array.

Error Codes

There are a number of error code sets. The one found most often is the *general* error code set. These are the errors returned in ErrCode. In addition, two other sets of error codes may be used by advanced programmers. These are the *socket* and *name* error code sets. These are accessed in the TCP'GET'CONNECTION'INFO call and provide information specific to the underlying *BSD Sockets* and *DNS Resolver* functions.

TCP.BSI stores text messages for all three error code sets. The error codes and messages are in the section "TAME Error Messages" near the end of this chapter.

Establishing Connections

To establish a connection to another program, use the following call:

```
XCALL TCP,TCP'CONNECT,ErrCode,Handle,Host,Port,SourcePort,Flags
```

SourcePort and ***Flags*** are optional. SourcePort is used only if you have a special application which must use a specific originating port number. If you don't include SourcePort, the system will pick a temporary port. If you wish to set Flags but not SourcePort, you may pass a SourcePort value of zero, which will behave the same as not including a SourcePort value. Possible values for ***flags*** are*:*

Value	Name	Description
1	TCP'CON'KEEPALIVE	Keepalives are disabled by default when using TCP'CONNECT; this flag enables them. Keepalives test the network connection periodically when no data is being passed. Using keepalives can prevent a program from waiting forever for input from a remote host that has been rebooted. This is usually needed only for background processes such as servers, rarely for clients which have a person controlling them. You may want to enable keepalives if you are developing a program that operates in the background and initiates connections. The down side of keepalives is that they can prevent a dialup connection from being dropped when open-but-quiet connections exist. An example of this is a user who establishes a connection, then goes home.
2	TCP'CON'NODELAY	By default, TCP'CONNECT enables delay; this flag disables it. This setting controls a TCP performance enhancement known as the “Nagle Algorithm.” Every packet of TCP data has at least 40 characters of control information added, even when sending a single character of data. Enabling delay causes TCP to allow only one tiny packet to be transmitted at a time, allowing extra time to buffer data for the next packet. Disabling delay allows tiny packets to be sent right away, which can hurt performance on a wide-area network if most traffic is small packets. Having delay enabled is good when passing keystrokes and echo, because there is a high probability that more characters will be available by the time the next packet is to be sent.
4	TCP'CON'FIRSTADDR	In some cases a single host name can have multiple IP addresses associated with it. By default, each address is tried in turn until a connection is established. Setting this flag causes TCP to try only the first address. If a host has only one address, this flag has no effect.

You can specify more than one flag by adding the values together.

Host is a string, and may contain a host name or a dotted decimal IP address. Likewise, ***Port*** is a string and may contain a port number or a service name defined in the TCP:SERVIC. file. When using service names you do not specify the protocol. In other words, “pop3” is correct, “pop3/tcp” is not.

We recommend you use a 4-byte binary value for ***Handle***. Upon return, this contains the handle for the new session.

Once the connection is established, or if it fails, you will get a TCP'SEVENT'CONNECTED or TCP'SEVENT'DROPPED event to notify you.

Accepting Connections - Part One

Writing a server to accept incoming connections requires a two-part procedure. First you *listen* for connection requests. As connections are requested, you *accept* them. To start listening for incoming connection requests, use the following call:

```
XCALL TCP,TCP'LISTEN,ErrCode,Handle,Port,Flags
```

Port is a string, and may contain a port number or a service name as listed in the TCP:SERVIC. file.

Flags is optional. Possible values are:

Value	Name	Description
1	TCP'LSN'GETNAME	When a new connection arrives the only identification available is the remote IP address. This flag asks TCP to resolve the address into a host name. You are not informed of the new connection until this process completes or times out. Looking up the IP address can take some time, especially if the connecting host is not local, or there are problems with the name servers. This can be very annoying to a connecting user, or even appear as if your host is down. Keep this in mind when deciding if you want to enable this feature.
2	TCP'LSN'FAILNAME	This flag enables the same feature at TCP'LSN'GETNAME. The difference is TCP'LSN'FAILNAME will drop the connection attempt if the lookup fails. Your application will only be notified of new connections with resolved host names.
4	TCP'LSN'NOKEEPALIVE	Keepalives are enabled by default when using TCP'LISTEN; this flag disables them. Keepalives test the network connection periodically when no data is being passed. Keepalives can prevent a program from waiting forever for input from a remote host that has been rebooted. This is usually desired for background processes such as servers, since system resources can be tied up indefinitely without keepalives. The down side of keepalives is that they can prevent a dialup connection from being dropped when open-but-quiet connections exist. An example of this is a user who establishes a connection, then goes home.

8	TCP'LSN'NODELAY	By default, TCP'LISTEN enables delay; this flag disables it. This setting controls a TCP performance enhancement known as the “Nagle Algorithm.” Every packet of TCP data has at least 40 characters of control information added, even when sending a single character of data. Enabling delay causes TCP to allow only one tiny packet to be transmitted at a time, allowing extra time to buffer data for the next packet. Disabling delay allows tiny packets to be sent right away, which can hurt performance on a wide-area network if most traffic is small packets. Having delay enabled is good when passing keystrokes and echo, because there is a high probability that more characters will be available by the time the next packet is to be sent.

You can specify more than one flag by adding the values together.

We recommend you use a 4-byte binary value for ***Handle***. Upon return, this contains the handle for the new session. This handle will never be used for an active connection. Instead, the session it identifies is always listening for new connections.

Accepting Connections - Part Two

When connection requests arrive, you are notified via the TCP’CONNECTED event. To accept the connection and begin servicing it, use the following call:

```
XCALL TCP,TCP’ACCEPT,ErrCode,Handle,NewHandle
```

Handle is the handle of the session that is listening; it informs you of the incoming connection. Upon return, ***NewHandle*** is filled in with a new handle to identify the new connection. The new handle will automatically receive a TCP’SEVENT’CONNECTED event of its own.

Sending Data

There are four primary rules governing the sending of data:

- Never send data unless you have received a TCP’SEVENT’DATAOUT (data out) event.
- Never perform multiple sends in response to a single data out event.
- Never modify or discard a data buffer until the next data out event. This is your indication that the data buffer is no longer being accessed.
- Never close a connection you have recently written to until the next data out event. This is your indication that the data has been moved from the data buffer.

Currently, the largest single write is 2048 bytes, To send larger amounts of data, you must perform multiple writes in separate data out events.

There are two ways to send data: fixed length and variable length. You can use fixed length to send data records as follows:

```
XCALL TCP,TCP'WRITE'RECORD,ErrCode,Handle,Record
```

Record is usually a data record defined using MAP statements. This call writes the entire length of the record out.

Variable-length writes can send text strings or such. Use this call:

```
XCALL TCP,TCP'WRITE'DATA,ErrCode,Handle,Buffer,Length
```

Buffer is the data you want to write, and ***Length*** is the number of characters.

Reading Data

You should only read data in response to a TCP'SEVENT'DATAIN event. Never attempt to read data without being notified.

There are three ways to read data: fixed length, variable length, and lines. The TCP protocol does not maintain record boundaries. The amount of data returned by a read will not usually match the amount of data written by each write. Because of this, two of the calls use a hidden event to actually move the data.

To read in a fixed-length record, use the following call:

```
XCALL TCP,TCP'READ'RECORD,ErrCode,Handle,Record
```

This call does not fill in the ***Record*** itself, but places the session in a mode where Record will be filled in using as many TCP transfers as required. Upon completion, a TCP'SEVENT'RECORD event is generated. At that time, the session reverts to normal mode and again begins generating TCP'SEVENT'DATAIN events.

To read in a variable-length amount of data, use this call:

```
XCALL TCP,TCP'READ'DATA,ErrCode,Handle,Buffer,Length
```

Currently available data will be written to ***Buffer***, to its total length. If less data is available, Buffer will be partially filled. The number of characters read is placed in ***Length***.

The amount of data available to read ***has nothing to do*** with individual writes at the other end. The TCP protocol itself can freely break apart or combine packets of data in transit. If you assume you can read a fixed length because of a known fixed-length write at the other end, your program may work fine on a local network, only to break in unexpected ways on a larger network.

A different call reads data a line at a time. This call reads up to the next linefeed, or up to the maximum length of the string if no linefeed is found. A null is added after the linefeed for proper null termination

of the string. If no carriage return is present, one is added before the linefeed. This call is also event driven, operating like the record-fill method above. You will receive a TCP'SEVENT'RECORD event when the line is ready. The call is:

```
XCALL TCP,TCP'READ'LINE,ErrCode,Handle,String
```

Closing Sessions

When you are finished with a session you need to close it. This includes sessions which are not connected, active connections, and remotely closed connections. This is an important step: it allows the resources from the session to be reclaimed. There are two ways to close a session: one preserves queued data and the other discards it. The first is used for normal operation, while the second can be used in case of error.

The first method uses this call to close a session and allow pending output to complete transmission:

```
XCALL TCP,TCP'CLOSE,ErrCode,Handle
```

The second method uses this call to quickly terminate a session and discard any pending data:

```
XCALL TCP,TCP'ABORT,ErrCode,Handle
```

The remote end is informed about the connection shutdown. If the remote is also using the TCP XCALL, it receives a TCP'SEVENT'DROPPED rather than a TCP'SEVENT'DISCONNECTED event. In the case of BSD Sockets programming, the remote will receive an ECONNRESET error.

Spawning Jobs

When developing applications which support many connections, it is useful to start another job to control a new session. Methods of control and notification are described below. To create a new job, use the following call:

```
XCALL TCP,TCP'START'SPAWNED'JOB,ErrCode,Pid,Memory,Command,Flags,Name
```

Pid is filled in with the process ID upon successful creation of the new job.

Memory specifies the number of bytes to give the job. When the job is created, approximately 6KB is added to this value for overhead.

Command is a string and contains the first command the job should execute when it is started.

Flags defines various options for the job as follows:

TCP'SPAWN'FAILNAME	Fail if jobname already exists
TCP'SPAWN'LOGOFF	Start job logged off (otherwise uses caller's login)
TCP'SPAWN'ALLOWDOT	Job may be at monitor level

Name is optional and specifies the desired name of the job and terminal. By default, the names created are TSKAAA through TSKZZZ. If the name exists, characters are incremented until a free name is found, unless the TCP'SPAWN'FAILNAME flag is set.

Terminating Spawned Jobs

Use the following call to terminate one of your program's spawned jobs :

```
XCALL TCP,TCP'TERMINATE'SPAWNED'JOB,ErrCode,Pid
```

This terminates the job in the usual AlphaTCP manner. That is, the job is sent an exit request software interrupt and a control-C. If the job does not exit within a few seconds, another is sent. Failing both attempts, the job is aborted using an abort software interrupt.

Important Cleanup

It is very important to clean up after spawned jobs terminate. Not doing so uses up important system resources. When a spawned job terminates, a TCP'GEVENT'CHILD global event occurs. In response to this, you must request terminating status with the following call:

```
XCALL TCP,TCP'CLEANUP'SPAWNED'JOB,ErrCode,Pid,Status
```

This call returns the process ID of the terminated job in ***Pid***, and the exit value (as stored in JOBERR) in ***Status***. You should perform this call in a loop until ***ErrCode*** returns TCP'ERR'ENRDY.

Attention Events

There is a mechanism for jobs to get each other's attention and to pass a limited amount of data to each other. You can use it to pass handles or other useful items to child jobs. Use the following call to request another job's attention:

```
XCALL TCP,TCP'SET'ATTENTION,ErrCode,Pid,Arg1,Arg2
```

Pid should be the process ID of the job to receive the attention event (the *target job*).

Arg1 and ***Arg2*** are optional. If used, they should be 4-byte binary values containing the data you wish to send.

The target job will receive a TCP'GEVENT'ATTENTION global event. In response, it should perform the following call:

```
XCALL TCP,TCP'GET'ATTENTION,ErrCode,Pid,Arg1,Arg2
```

Pid contains the process ID of the job sending the attention.

Arg1 and ***Arg2*** are optional. If used, they should be 4-byte binary values; they are filled in with the sender's passed arguments.

The event and arguments are buffered, allowing the sender to continue running without waiting for the receiver to process the event. The attention mechanism is a handy way to hand off session control to a child.

Transfer of Control

When a job spawns a child to service a connection, control of the session must be transferred from the parent to the child. You can pass the handle to the child via the command line or using attention events, as described in the previous section. After this, the child must request control of the session from the parent. The following call does this:

```
XCALL TCP,TCP'REQUEST'CONTROL,ErrCode,Handle,NewHandle
```

Handle should be the handle received from the parent. Upon return, ***NewHandle*** is filled in with a handle the job may use to control the session.

When the request call is made, the parent receives a TCP'SEVENT'HANDOFF event on the session. In response, the parent job should perform the following call:

```
XCALL TCP,TCP'RELEASE'CONTROL,ErrCode,Handle
```

Once this transaction takes place, the handle originally passed to the child becomes invalid for both the child and parent. Also, the request call waits until the parent performs the release; therefore the parent should waste no time releasing the session.

Path and Filename Conversion

With the large difference in filename conventions from AMOS to many other operating systems, a method is needed to translate names to AMOS format. A common routine is used throughout AlphaTCP. This routine is accessible via the following call:

```
XCALL TCP,TCP'CONVERT'FILENAME,ErrCode,Path,Filename,Flags
```

Path is a string containing the foreign path or filename.

Filename is a string to receive the converted file specification.

Flags controls processing of the path. Currently, the only flag available is TCP'PATH'NODIR. This causes all path information to be discarded, leaving just a filename. By default, the path will be processed in an attempt to create meaningful ersatz, device, and account values.

The *AlphaTCP User's Guide* contains a detailed explanation of the processing performed on the name. Forward slashes are assumed to be path delimiters. Before converting a DOS path, you must convert all backslashes.

Connection Information

The following call retrieves a number of details regarding a connection:

```
XCALL TCP,TCP'GET'CONNECTION'INFO,ErrCode,Handle,LocalName,LocalAddr,
      LocalPort,RemoteName,RemoteAddr,RemotePort,Distance,ErrSocket,ErrName
```

LocalName and ***RemoteName*** are strings which receive the host name of each system if it can be determined.

LocalAddr and ***RemoteAddr*** are strings which receive the dotted decimal IP address of the system at each end.

LocalPort and ***RemotePort*** are the port numbers in use at each end of the connection.

Distance represents an analysis of the local and remote addresses, and may have one of the following values:

TCP'IS'LOCALHOST	Other job running on same system as this one
TCP'IS'LOCALNET	Other job on a system on the local network
TCP'IS'REMOTE	Other job is located on another network

ErrSocket contains the last *BSD Sockets* error found. For a list of these errors, see the section "TAME Error Messages" near the end of this chapter.

ErrName contains the last *DNS Resolver* error found. For a list of these errors, see the section "TAME Error Messages" near the end of this chapter.

Setting the Alarm Clock Timer

The alarm clock timer can set time-outs or perform a function at set intervals. Only one alarm may exist at a time. The following call starts the timer:

```
XCALL TCP,TCP'SET'ALARM,ErrCode,Ms
```

Ms is the number of milliseconds the timer will run before expiring. A value of zero stops the timer.

Trapping SIEXIT Requests

When an application which spawns jobs, such as AlphaTCP, wants to terminate an application, it sets a special interrupt and sends a Control-C. If your application wants to distinguish between this and a user-entered Control-C, you need to detect the special interrupt. The special interrupt is known as SIEXIT. Once trapped, SIEXIT will be returned as an activity flag. The call to set up SIEXIT processing is:

```
XCALL TCP,TCP'TRAP'SIEXIT,ErrCode,Action
```

Action tells the call what to do. You can use the following values:

TCP'SIEXIT'CLEARTRAP	Disable SIEXIT processing
TCP'SIEXIT'TRYTRAP	Enable unless already handled externally
TCP'SIEXIT'SETTRAP	Enable SIEXIT processing

The error code TCP'ERR'ENRDY is returned if TCP'SIEXIT'TRYTRAP determines that SIEXIT is already handled externally.

Since a Control-C is also processed during an SIEXIT, your application will need to set up various ON ERROR and RESUME conditions. These need to ensure that TCP'WAIT'FOR'ACTIVITY is called to set the flags. Also, you should avoid calling TCP'WAIT'FOR'ACTIVITY a second time if the Control-C occurs within TCP'WAIT'FOR'ACTIVITY.

QUICK REFERENCE LIST OF CALLS

The following alphabetic list includes the format and a brief description for all uses of XCALL TCP. To save space, we've left the `XCALL TCP,` off the beginning of each call:

Calling Format	Description
`TCP'ABORT,ErrCode,Handle`	Terminate a session. Pending data is thrown away.
`TCP'ACCEPT,ErrCode,Handle,NewHandle`	Accept incoming connection request. Start session identified by ***NewHandle*** to service it.
`TCP'CLEANUP'SPAWNED'JOB,ErrCode,Pid,Status`	Clean up after terminating spawned job. Must be done to reclaim job's resources.
`TCP'CLOSE,ErrCode,Handle`	Terminate a session after processing pending data.
`TCP'CONNECT,ErrCode,Handle,Host,Port, SourcePort, Flags`	Request connection to ***Host***.
`TCP'CONVERT'FILENAME,ErrCode,Path,Filename, Flags`	Convert filename into AMOS format.
`TCP'EVENTS'DONE,ErrCode,Handle`	Notify TCP that you have finished processing all current events.
`TCP'GET'ALL'EVENTS,ErrCode,Gevents, EventArray,FillCount`	Get global events and events for all sessions you control.
`TCP'GET'ATTENTION,ErrCode,Pid,Arg1,Arg2`	Acknowledge request for attention and accept data from other job (usually parent)
`TCP'GET'CONNECTION'INFO,ErrCode,Handle, LocalName,LocalAddr,LocalPort,RemoteName, RemoteAddr,RemotePort,Distance,ErrSocket, ErrName`	Retrieve information about TCP connection identified by ***Handle***.
`TCP'GET'EVENTS,ErrCode,Handle,Gevents, Sevents`	Get global events and events for the session identified by ***Handle***.
`TCP'LISTEN,ErrCode,Handle,Port,Flags`	Establish session to start listening for incoming connection requests.
`TCP'QUERY'EVENTS,ErrCode,HaveEvents,AutoSet`	Check for pending TCP events.
`TCP'READ'DATA,ErrCode,Handle,Buffer,Length`	Read variable-length data.

Calling Format	Description
`TCP'READ'LINE,ErrCode,Handle,String`	Read data up to next linefeed character.
`TCP'READ'RECORD,ErrCode,Handle,Record`	Read fixed-length data record
`TCP'RELEASE'CONTROL,ErrCode,Handle`	Release control of session identified by ***Handle*** to child job.
`TCP'REQUEST'CONTROL,ErrCode,Handle, NewHandle`	Request control of session identified by ***Handle*** from parent and change handle to ***NewHandle***.
`TCP'SET'ALARM,ErrCode,Ms`	Set alarm clock timer.
`TCP'SET'ATTENTION,ErrCode,Pid,Arg1,Arg2`	Request another job's (usually a child) attention and pass data to it.
`TCP'START'SPAWNED'JOB,ErrCode,Pid,Memory, Command,Flags,Name`	Spawn a child job.
`TCP'TERMINATE'SPAWNED'JOB,ErrCode,Pid`	Terminate spawned job.
`TCP'TRAP'SIEXIT,ErrCode,Action`	Trap special interrupt (SIEXIT) sent to spawned job.
`TCP'WAIT'FOR'ACTIVITY,ErrCode,Activity`	Have job wait for activity.
`TCP'WRITE'DATA,ErrCode,Handle,Buffer,Length`	Write variable-length data
`TCP'WRITE'RECORD,ErrCode,Handle,Record`	Write fixed-length data record.

TAME ERROR MESSAGES

As mentioned in the "Error Codes" section above, there are three sets of error codes related to the TCP XCALL:

- The *general* error codes may be returned in the ErrCode variable in any TCP call. The TCP.BSI file defines equates for the general error codes so you can test for them easily in your application.
- The *socket* error codes contain information about the BSD socket function, and can be accessed using the TCP'GET'CONNECTION'INFO call.
- The *name* error codes are also accessed using TCP'GET'CONNECTION'INFO. They contain information about the *DNS Resolver* functions.

The following code fragment demonstrates conversion between an error code and the string representation found in TCP.BSI. The *xxx* in the code may be replaced by the words GENERAL, SOCKET, or NAME. The error code is passed in TCP'xxx'ERROR'CODE and the string is returned in TCP'xxx'ERROR'TEXT:

```
MAP1 I,F,6
MAP1 TCP'xxx'ERROR'CODE,F,6
MAP1 TCP'xxx'ERROR'TEXT,S,100

CONVERT'TCP'xxx'ERROR:
   TCP'xxx'ERROR'TEXT=""
   FOR I = 1 TO TCP'xxx'ERROR'MAX
      IF TCP'xxx'ERROR'IDX(I)=TCP'xxx'ERROR'CODE THEN &
         TCP'xxx'ERROR'TEXT=TCP'xxx'ERROR'STRING(I) : I=TCP'xxx'ERROR'MAX
   NEXT I
   IF TCP'xxx'ERROR'TEXT="" THEN TCP'xxx'ERROR'TEXT="Unknown error code"
   RETURN
```

General Error Codes

Value	Equate Variable	Text Message
1	TCP'ERR'ENAVL	TCP interface not available
2	TCP'ERR'ENVLD	Invalid TCP function call
3	TCP'ERR'EINTR	TCP function interrupted
4	TCP'ERR'ENRDY	Not ready for requested TCP operation
5	TCP'ERR'EDISA	TCP session has been disabled
6	TCP'ERR'ESOOR	TCP service out of resources
7	TCP'ERR'ESOOM	TCP service out of memory
8	TCP'ERR'ESOCK	General TCP socket failure
9	TCP'ERR'EPORT	Invalid TCP service or port
10	TCP'ERR'EHOST	Invalid TCP host name
11	TCP'ERR'EHUNR	TCP Host unreachable or not responding
12	TCP'ERR'EPARM	Parameter error in TCP call
13	TCP'ERR'ETDV	Spawn failure, cannot find tdv
14	TCP'ERR'EIDV	Spawn failure, cannot find idv
15	TCP'ERR'EMEM	Spawn failure, not enough memory
16	TCP'ERR'EFUL	Spawn failure, job table is full
17	TCP'ERR'EDUP	Spawn failure, job already exists
18	TCP'ERR'ELIC	Spawn failure, would exceed AMOS license

Socket Errors

Value	Text Message	Value	Text Message
1	File specification error	523	Address family not supported
2	Insufficient free memory	524	Socket type not supported
3	File not found	525	Protocol not supported
4	File already exists	526	No buffer space available
6	Device full	528	Socket is already connected
7	Device error	529	Socket is not connected
8	Device in use	530	Bad protocol option
10	Protection violation	531	Connection reset by peer
11	Write protected	532	Software caused connection abort
12	File type mismatch	533	Network is down
13	Device does not exist	534	Connection refused

Value	Text Message	Value	Text Message
14	Illegal block number	535	Host is unreachable
27	Remote is not responding	536	Protocol wrong type for socket
28	File in use	537	Operation not supported on socket
30	Deadly embrace possible	538	IP subnet table full
500	Device not a stream	539	Subnet module not linked
501	no data	540	Unknown ioctl call
502	timer expired	541	Failure in Streams buffer allocn
503	out of streams resources	542	ICMP Protocol unreachable
504	Machine is not on the network	543	ICMP Port unreachable
505	Package not installed	544	ICMP Network unreachable
506	The object is remote	545	Protocol family not supported
507	the link has been severed	546	Can't send after socket shutdown
508	advertise error	547	Network dropped connection on reset
509	srmount error	548	Destination address required
510	Communication error on send	549	Invalid Ethernet Packet
511	Protocol error	550	Type registration error
512	multihop attempted	551	illegal address
513	Inode is remote	552	message too big for buffer
514	Cross mount point	553	no message of requested type exists
515	trying to read unreadable message	554	File table overflow
516	given log. name not unique	1000	USAM error
517	f.d. invalid for this operation	1001	Domain error
518	Remote address changed	1002	Range error
519	try again	1003	interrupted system call
520	Socket operation on non-socket	1004	too many open files
521	Can't assign requested address	1005	no such device or address
522	Address already in use	1006	not super user

Name (DNS Resolver) Errors

Value	Text Message
01	Unknown host
02	Host name lookup failure
03	Unknown server error
04	No address associated with name

PROGRAMMING SAMPLES

You can find sample programs which use the TCP XCALL in DSK0:[7,6] (BAS:). There are two samples: TCPSVR.BAS is a sample server program; TCPCLI.BAS is a sample client.

Appendix A - Quick Reference

All XCALL subroutine files must have a .SBR extension. The subroutines supplied with your system software reside in account [7,6] of the System Disk (DSK0:). Some of the subroutines must be loaded into either system or user memory before they are used. To load a subroutine into user memory:

```
LOAD DSK0:xxxxxx.SBR[7,6]RETURN
```

To load a subroutine into system memory, enter this statement in your system initialization command file:

```
SYSTEM DSK0:xxxxxx.SBR[7,6]
```

ACCEPT

PURPOSE: Allows you to accept one character of arbitrary data from the terminal, including control characters and others normally trapped by the system.

SYNTAX: XCALL ACCEPT,char

NOTES: Your program MUST call the NOECHO subroutine first.

BASORT

PURPOSE: Sorts both random and sequential files.

LOAD: AMSORT.SYS into system or user memory.

SYNTAX: The random file syntax is:

```
XCALL BASORT,channel,rec'count,rec'siz, &
  key1'siz, key1'pos,key1'order,key2'siz, &
  key2'pos,key2'order, key3'siz, key3'pos, &
  key3'order,key1'type,key2'type, key3'type
```

Key types: 0 = String
1 = Floating Point
2 = Binary

The sequential file syntax is:

```
XCALL BASORT,in'channel,out'channel, recsiz,&
  key1'siz, key1'pos,key1'order,key2'siz, &
  key2'pos,key2'order, key3'siz,key3'pos,key3'order
```

NOTES: BASORT.SBR is not re-entrant, so you ***must not*** load BASORT.SBR into system memory. Arguments must NOT be in binary format.

COMMON

PURPOSE: Allows you to place data into a common storage area of either user or system memory.

LOAD: COMMON.SBR into system or user memory.

SYNTAX: `XCALL COMMON,send/receive,msgnam,variable(s)`

EXAMPLE: `XCALL COMMON,SEND,FILE'NAME,MESSAGE'TO'BE'SENT`

NOTES: First define binary variables that signal ***send*** or ***receive***. If receive, define a confirmation variable:

```
MAP1  SEND,B,1,0
MAP1  RECEIVE
      MAP2  F'RCV,B,1,1
      MAP2  RCVFLG,B,1,0
```

If loading into system memory, ***msgnam*** must be unique.

ECHO

PURPOSE: Turns on the terminal echoing (normal display) mode.

SYNTAX: `XCALL ECHO`

FLOCK

PURPOSE: Allows you to place "locks" on files and records.

LOAD: FLOCK.SBR—MUST be in System Memory

SYNTAX: `XCALL FLOCK,action,mode,return-code,file,record`

EXAMPLE: `XCALL FLOCK,0,2,RET,1001`

NOTES: See Chapter 4 for actions, modes, and codes.

FNDUSR

PURPOSE: Returns information about a user name.

SYNTAX: `XCALL FNDUSR,user'name,status,user'map`

NOTES: If ***user'name*** has a length of 0, uses the caller's user name. ***User'map*** contains the returned information. See Chapter 12 for its structure and content.

GTLANG

PURPOSE: Allows you to access the contents of the language definition table currently defined for your job.

SYNTAX: `XCALL GTLANG,status,gtlang'map`

NOTES: The ***gtlang'map*** is a map structure that defines variables to contain specific information about the current language. See Chapter 8.

IDTIM

PURPOSE: Converts a date and/or a time from string to internal format.

SYNTAX: `XCALL IDTIM,date,var'for'date,var'for'time,flag,status`

NOTES: ***Flag*** can be: 1 = Don't scan for date, 2 = Don't scan for time. If both the date and the time are specified, the time must follow the date in the string. The time string format is: `hh:mm{{:ss} {{am}{{pm}`

LKINFO

PURPOSE: Returns user and terminal information of a job which has a record locked under LOKSER.

SYNTAX: `XCALL LKINFO file, status, jobname, termname, jobuser`

NOTES: ***File*** can be either a channel number or file name. See Chapter 11 for possible returned status codes.

NOECHO

PURPOSE: Turns off the terminal's echoing, and puts terminal into image mode.

SYNTAX: `XCALL NOECHO`

ODTIM

PURPOSE: Converts date and/or time from internal format to string.

SYNTAX: `XCALL ODTIM,var'for'decoded,date,time,flags`

NOTES: See chapter 11 for the flags.

RENAME

PURPOSE: Allows you to rename disk files from within AlphaBASIC.

SYNTAX: `XCALL RENAME,oldnam,newnam,status`

SLEEP

PURPOSE: Puts your program to sleep for a specified number of seconds.

SYNTAX: `XCALL SLEEP,amount`

SPOOL

PURPOSE: Allows you to send a file to a printer.

SYNTAX: `XCALL SPOOL,file$,spool'table`

`SPOOL'TABLE is a mapped variable with the form:`

```
MAP1  SPOOL'TABLE
      MAP2  PRINTER,S,6,""      ; Printer spec
      MAP2  CPU,F,6,0           ; Network cpuID
      MAP2  SWITCHES,F,6,0      ; Switches
      MAP2  COPIES,F,6,2        ; # Copies to print
      MAP2  BANNER,S,45,""
      MAP2  LPP,F,6,0           ; Lines per page
      MAP2  WIDTH,F,6,0         ; Width of page
      MAP2  FORMS,S,6,""        ; Print form
      MAP2  PRI,F,6,0           ; Priority
      MAP2  AFTER'DATE,B,4,0    ; Print after date
      MAP2  SEP'DATE,@AFTER'DATE
            MAP3  MONTH,B,1
            MAP3  DAY,B,1
            MAP3  YEAR,B,1
      MAP2  AFTER'TIME,B,4,0    ; Print after time
      MAP2  SEP'TIME,@AFTER'TIME
            MAP3  HOURS,B,1
            MAP3  MINUTES,B,1
            MAP3  SECONDS,B,1
      MAP2  RESTART,F,6,0       ; Restart yes/no
      MAP2  START,F,6,0         ; Start at page
      MAP2  FINISH,F,6,0        ; End at page
      MAP2  LIMIT,F,6,0
```

STRIP

PURPOSE: Removes trailing blanks from a string.

SYNTAX: `XCALL STRIP,string`

EXAMPLE: `XCALL STRIP,TEXT'LINE$`

TCP

PURPOSE: Gives AlphaBASIC access to TCP features.

SYNTAX: `XCALL TCP,…`

Many formats are available to perform various TCP functions. See Chapter 13.

TRMCHR

PURPOSE: Returns terminal characteristics.

SYNTAX: `XCALL TRMCHR,status,trmchr'map`

Trmchr'map is:

```
MAP1  TRMCHR'MAP
      MAP2  TRMCHR'FLAGS,F,6
      MAP2  TRMCHR'ROWS,F,6
      MAP2  TRMCHR'COLS,F,6
      MAP2  TRMCHR'COLORS,F,6
      MAP2  TRMCHR'FORE,F,6
      MAP2  TRMCHR'BACK,F,6
      MAP2  TRMCHR'WINROW,F,6
      MAP2  TRMCHR'WINCOL,F,6
```

XLOCK

PURPOSE: Allows you to set and test device and file "locks."

LOAD: XLOCK.SBR ***must*** be in system memory

SYNTAX: `XCALL XLOCK,mode,lock1,lock2`

mode may be:

0: Set lock and return
1: Set lock (Wait if locked; then set)
2: Clear lock (if set by your job)
3: Return a list of all system locks and the jobs that set them.

EXAMPLE: `XCALL XLOCK,MODE,MYJOB,LISTARRAY`

NOTES: Your AlphaBASIC program must first set up:

```
MAP1  MODE,B,2
MAP1  LOCK1,B,2
MAP1  LOCK2,B,2
```

Define parameters on even-byte memory boundaries.

XMOUNT

PURPOSE: Allows you to mount a disk from within an AlphaBASIC program without leaving AlphaBASIC.

SYNTAX: `XCALL XMOUNT,device{/U},volume'id`

EXAMPLES:

```
XCALL XMOUNT,"DDA1:"
XCALL XMOUNT,"DDA4:/U"
XCALL XMOUNT,"FLP1:",VOLUME'ID
```

Appendix B - Error Messages

?AMSORT.SYS not found in memory

The sort utility routine, AMSORT.SYS, must be loaded into user or system memory before calling BASORT.SBR.

?Bad channel number in XCALL BASORT

The channel number you passed to BASORT was invalid. This error can occur if the file is not open, or if the value given as channel is not an integer.

?Bad DEVICE specification

The device you specified as the printer in your DEVICE command (as part of a SPOOL XCALL) is not a valid device specification. Make sure the device you specified is in the device table (the DEVTBL line of your system initialization command file), and the device has a driver program in DSK0:[1,6].

?Cannot sort files opened in RANDOM'FORCED mode

You can't sort a file opened in RANDOM'FORCED mode. If you need to sort this file, close it and reopen it without using RANDOM'FORCED.

?File improperly open in XCALL BASORT

When you call BASORT, the file you wish to sort must be open for INPUT or RANDOM processing.

?Illegal command X

You used the SPOOL XCALL, and the subroutine found an error in your spooler parameter file when it tried to send the file to the spooler. "X" defines the error. Check the spelling in the file for errors, and make sure your parameter command arguments are in the proper form.

?Illegal value in XCALL BASORT

One of the arguments to the BASORT call was invalid. Check the key sizes and positions to make sure they fit into the record size which you specified. Also make sure you have given valid key types.

?Improper argument format for xxxxxx.SBR

The number, type, and length of the arguments passed to the subroutine ***xxxxxx*** is not correct. Check the syntax of your XCALL statement and the documentation for the subroutine and make sure you are using the correct arguments.

?Read file error in XCALL BASORT

An error occurred during a read operation while sorting your file. Check your program to see if you are reading the correct data type, or see your System Operator for help.

?Specified operator not found

The job you specified in your OPERATOR command does not exist. Check your spelling, and examine a SYSTAT display to see a list of the valid jobs on the system.

?Write file error in XCALL BASORT

An error occurred during a write operation while sorting your file. Check your program to see if you are writing the correct data type, or see your System Operator for help.

?Wrong record size in XCALL BASORT

The record size you specified when calling BASORT does not match the record size you specified when you OPENed the file. BASORT will not accept a record size of 0. The maximum record sizes are 510 bytes for sequential files and 512 bytes from random files.

Appendix C - Glossary

DEADLOCK	A situation where two programs are each trying to access a file locked by the other. Neither program can proceed, and the “deadlock” cannot be broken, except by a manual reset of the system.
DELIMITER	An alphanumeric character that marks a division between two pieces of data. For example, in the date display 12/04/86, the slash “/” is a delimiter. A dash “-” is also a common delimiter.
ECHO	A response by your terminal to what you press on the keyboard. If you press a key (say the “a”), and you see a response on your terminal screen (an “a” appears), then your terminal is in ECHO mode (the normal way for it to be). Sometimes, for instance—when you are entering a password, the terminal echoing can be turned off, so what you type goes to the computer, but is not displayed on the screen.
FILE LOCKING	A process by which a file being used by one program is “locked”, so any other program that tries to access that file will be told the file is in use. This prevents problems that can occur if two programs were to work on the same file at the same time.
FLOATING POINT NUMBER	A “real” number. That is, a number that may contain a decimal part. For example, 4.175.
INTEGER NUMBER	A “whole” number. That is, a number that does not have a decimal part. For example, 3.
LABEL	A name that marks a place in a program, so other lines in the program can transfer control to the spot. For example, if you had a subroutine that added up three numbers, you might label it ADD’THREE’NUMS:. The label may not contain spaces, and must have a colon “:” after it (though you do not use the colon when referring to the label).
MOUNT	A way of preparing a disk device for use.
NULL VALUE	A variable expression that has no value. For a number, the value would be zero (0). For a string variable, it would be empty quotation marks (““).
QUEUE	A “waiting list”. For example, when printing files, a queue is used so the files sent to the printer do not print all at the same time, but rather in an orderly way—first come, first served.
RECORD	A piece of data. For instance, if you think of a data file as a bookcase, each “record” would be like a book—a separate unit of data contained within the file.

SPOOLING — A method of sending a file to a printer queue, so it can await printing.

SUBROUTINE — A section of program code that operates as a unit and does a task. The main program "calls" the subroutine, and, when it has done its work, it returns control back to the next line of the main program.

SWITCH — An option that can be added to a command.

SYSTEM MEMORY — An area of memory available to all of the users of the computer. Any file loaded into this memory area (by use of the SYSTEM command within the system initialization command file) can be used by any person or program. The advantage of using system memory is that, if a file is used regularly by many users, each user does not have to load a copy of that file into his or her own user memory. This saves both time and memory space.

USER MEMORY — An area of memory available to one user. When you LOAD a file or program into your user memory area, only you and your programs can access that file or program.

WILDCARD — A symbol that can represent a range of other characters. You might want to think of it as a joker in a deck of cards. For example, if you wanted to erase three files: FILE01.DAT,FILE02.DAT, and FILE03.DAT, you could type all this:

```
ERASE FILE01.DATA [RETURN]
ERASE FILE02.DAT [RETURN]
ERASE FILE03.DAT [RETURN]
```

or you could type:

```
ERASE FILE0?.DAT [RETURN]
```

where the question mark symbol "?" represents all characters, or:

```
ERASE *.DAT [RETURN]
```

where the asterisk "*" represents all combinations of characters (in this case, all file names). Of course, you would not use the second command if you had other .DAT files you wanted to keep.

XCALL — An AlphaBASIC command that runs a subroutine external to (outside) the current BASIC program. The subroutines XCALL can call are useful programs many people use, and thereby save you the time and trouble of writing such a subroutine yourself.

Document History

Revision	Release	Date	Description
A00	AMOS/L 1.3	3/85	New document, adapted from the *AlphaBASIC XCALL Subroutine User's Manual*, DSS-10008-00. Manual was divided into AMOS and AMOS/L versions. In addition, it was extensively rewritten to make it easier to read and access information.
A01	AMOS/32 1.0	7/86	Added information on the FNDUSR call.
A02	AMOS/L 2.0	3/88	Updated SPOOL to fully access the Task Manager Print Spooler.
DSO-00066-00, Rev. 00	AMOS/L 2.0A	9/88	Reformatted and reprinted. Also corrected errors and expanded examples.
01	AMOS 2.2	3/91	Added MSGLOG in Chapter 11.
02	AMOS 2.3	9/96	Added LKINFO in Chapter 11. New error message, new flag for ODTIM.
03	AMOS 2.3A, PR 11/97	11/97	Added Chapter 13 for TCP XCALL. Changes to IDTIM and ODTIM in Chapter 11, and to FNDUSR in Chapter 12. Added FNDUSR and TCP and corrected error in ODTIM in Appendix A.

Index

.SBR extension · 1-3
5-tuple · 13-1

A

ACCEPT · 10-1
Action and mode · 4-2
AMSORT.SYS · 2-2
Arbitrary data · 10-1

B

BASORT · 2-1

C

Clear lock mode · 5-4
Client/server programming · 13-1
COMMON · 3-1
    variable length · 3-1
Common data · 3-1
Common storage · 3-1
COPIES option (SPOOL) · 6-2
Currently defined language · 8-3

D

Data packet · 3-1
Date conversion · 11-1, 11-2
Deadlock · 4-7
DEL command · 1-3
Detecting TCP events · 13-5
Disk block · 5-6
Domain names · 13-2

E

ECHO · 10-1
Event-driven programming · 13-3
Exclusive use · 4-2

F

File lock · 5-1
FILE option (SPOOL) · 6-1
File record · 5-6
File-open interlock · 4-6
FLOCK · 4-1
FNDUSR · 12-1
FORM option (SPOOL) · 6-2

G

Global events · 13-7
Graphics conventions · 1-4
GTLANG · 8-1
GTLANG'MAP · 8-1

H

Host names · 13-2

I

IDTIM · 11-1
Image mode · 10-1
Information on system users · 12-1
Interconsistency problem · 4-2
IP addresses · 13-1

K

Key (BASORT) · 2-1

L

Label · 1-2
Language definition table · 8-1
List lock mode · 5-4
LKINFO · 11-7
LOAD command · 1-3, 2-2, 2-6, 3-1, 5-2, 7-1
Lock · 5-3
Lock and return mode · 5-4
Lock and wait mode · 5-4
Logical record · 5-6
LPP option (SPOOL) · 6-2

M

Machine language · 1-1
Manual organization · 1-1
MAP statement · 1-2, 3-4, 5-2

Memory-based sort · 2-1
Mode (XLOCK) · 5-2
Mounting a disk · 7-1
MSGLOG · 11-6
Multiple update problem · 4-1
Multi-user file access · 4-1

N

NOECHO · 10-1
Non-exclusive use · 4-2

O

ODTIM · 11-2

P

Port numbers · 13-1
Preventing deadlock · 4-7
PRINTER option (SPOOL) · 6-2
Procedural programming · 13-3
Process IDs · 13-6
Program chaining · 3-1
Protocol · 13-2

R

Random file · 4-3
Record-update interlock · 4-6
Removing trailing blanks · 11-6
RENAME · 11-5
Reserved file-channel number · 4-4
Return-code · 4-2, 4-4

S

Send message to system log · 11-6
Sequential file · 4-3
Server programs · 13-1
Service names · 13-2
Session events · 13-7
SETUSI · 13-6
SIEXIT · 13-16
SLEEP · 11-6
Sorting random files · 2-2
Sorting sequential files · 2-5
Spawning TCP jobs · 13-13, 13-14
SPOOL · 6-1
  acronym · 6-1
  examples · 6-3
STRIP · 11-6
SWITCHES numeric code (SPOOL) · 6-2
SWITCHES option (SPOOL) · 6-2
SYSTEM command · 1-3
System memory · 1-3
System user information · 12-1

T

TAME · 13-1
TCP · 13-1
  5-tuple · 13-1
  accepting a connection · 13-10, 13-11
  alarm clock · 13-16
  cleanup · 13-14
  client/server programming · 13-1
  closing a session · 13-13
  connections · 13-8, 13-10, 13-11
  description · 13-1
  detecting events · 13-5
  DNS resolver errors · 13-20
  domain names · 13-2
  error messages · 13-18
  errors · 13-8
  event-driven programming · 13-3
  events · 13-5, 13-7
  file name conversion · 13-15
  getting attention · 13-14
  getting connection information · 13-16
  global events · 13-7
  handles · 13-6
  host names · 13-2
  include file · 13-5
  IP addresses · 13-1
  list of calls · 13-17
  making a connection · 13-8
  operation · 13-4
  port numbers · 13-1
  procedural programming · 13-3
  process IDs · 13-6
  programming overview · 13-3
  protocol · 13-2
  reading data · 13-12
  sample programs · 13-20
  sending data · 13-11
  service names · 13-2
  session events · 13-7
  SETUSI · 13-6
  SIEXIT · 13-16
  socket errors · 13-19
  spawning jobs · 13-13, 13-14
  terminating an application · 13-16
  timer · 13-16
  transferring control · 13-15
  using · 13-4
  variable types · 13-4
TCP.BSI file · 13-5
TCP’CONNECTED · 13-11
TCP’CONVERT’FILENAME · 13-15
TCP’EVENTS’DONE · 13-8
TCP’GET’ALL’EVENTS · 13-8
TCP’GET’ATTENTION · 13-14

TCP’GET’CONNECTION’INFO · 13-16
TCP’GET’EVENTS · 13-7
TCP’LISTEN · 13-10
TCP’READ’DATA · 13-12
TCP’READ’LINE · 13-12
TCP’READ’RECORD · 13-12
TCP’SET’ATTENTION · 13-14
TCP’WAIT’FOR’ACTIVITY · 13-5
TCP’WRITE’DATA · 13-12
TCP’WRITE’RECORD · 13-12
TCP'ABORT · 13-13
TCP'CLEANUP'SPAWNED'JOB · 13-14
TCP'CLOSE · 13-13
TCP'CONNECT · 13-8
TCP'QUERY'EVENTS · 13-5
TCP'RELEASE'CONTROL · 13-15
TCP'REQUEST'CONTROL · 13-15
TCP'SET'ALARM · 13-16
TCP'START'SPAWNED'JOB · 13-13
TCP'TERMINATE'SPAWNED'JOB · 13-14
TCP'TRAP'SIEXIT · 13-16
Terminal characteristics · 9-1
Terminal input and display · 10-1
Time conversion · 11-1, 11-2
Time format · 11-2
TRMCHR · 9-1
TRMCHR’FLAGS · 9-2
TRMCHR’MAP · 9-1

U

User map · 12-1
User map fields · 12-2
User memory · 1-3
User name · 12-1
User status · 12-1
Using XCALL subroutines · 1-3

V

Variable · 1-2

W

Where system looks for subroutines · 1-3
WIDTH option (SPOOL) · 6-2

X

XCALL · 1-1
  search sequence · 1-3
XLOCK · 5-1
  modes · 5-2, 5-3
  record number calculation · 5-6
  wildcards · 5-5
XMOUNT · 7-1
  unmount option · 7-2